

대분류/20
정보통신

중분류/01
정보기술

소분류/02
정보기술개발

세분류/02
응용SW엔지니어링

능력단위/31

NCS학습모듈

프로그래밍 언어 활용

LM2001020231_19v4



교육부

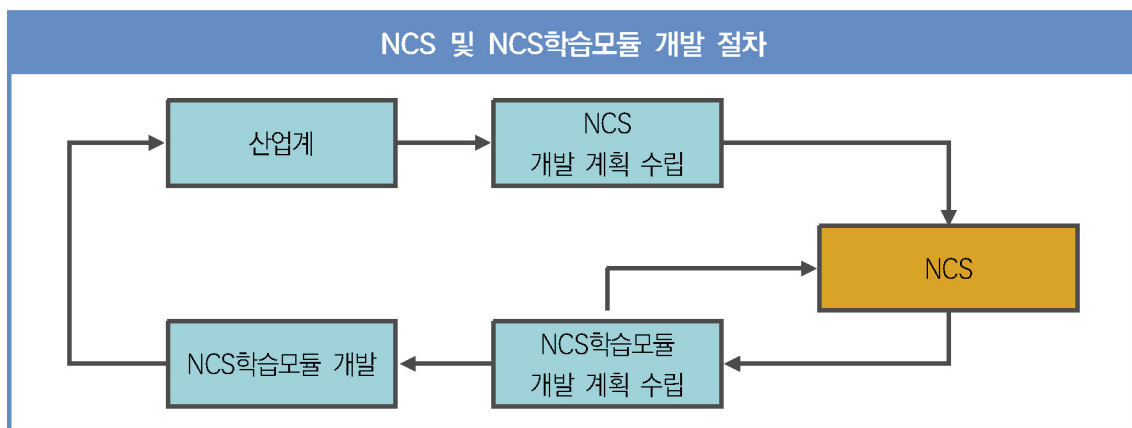
NCS 학습모듈은 교육훈련기관에서 출처를 명시하고 교육적 목적으로 활용할 수 있습니다. 다만 NCS 학습모듈에는 국가(교육부)가 저작권 일체를 보유하지 않은 저작물들(출처가 표기되어 있는 도표, 사진, 삽화, 도면 등)이 포함되어 있으므로 이러한 저작물들의 변형, 복제, 공연, 배포, 공중 송신 등과 이러한 저작물들을 활용한 2차 저작물의 생성을 위해서는 반드시 원작자의 동의를 받아야 합니다.

NCS학습모듈의 이해

※ 본 NCS학습모듈은 「NCS 국가직무능력표준」사이트(<http://www.ncs.go.kr>) 에서 확인 및 다운로드할 수 있습니다.

I NCS학습모듈이란?

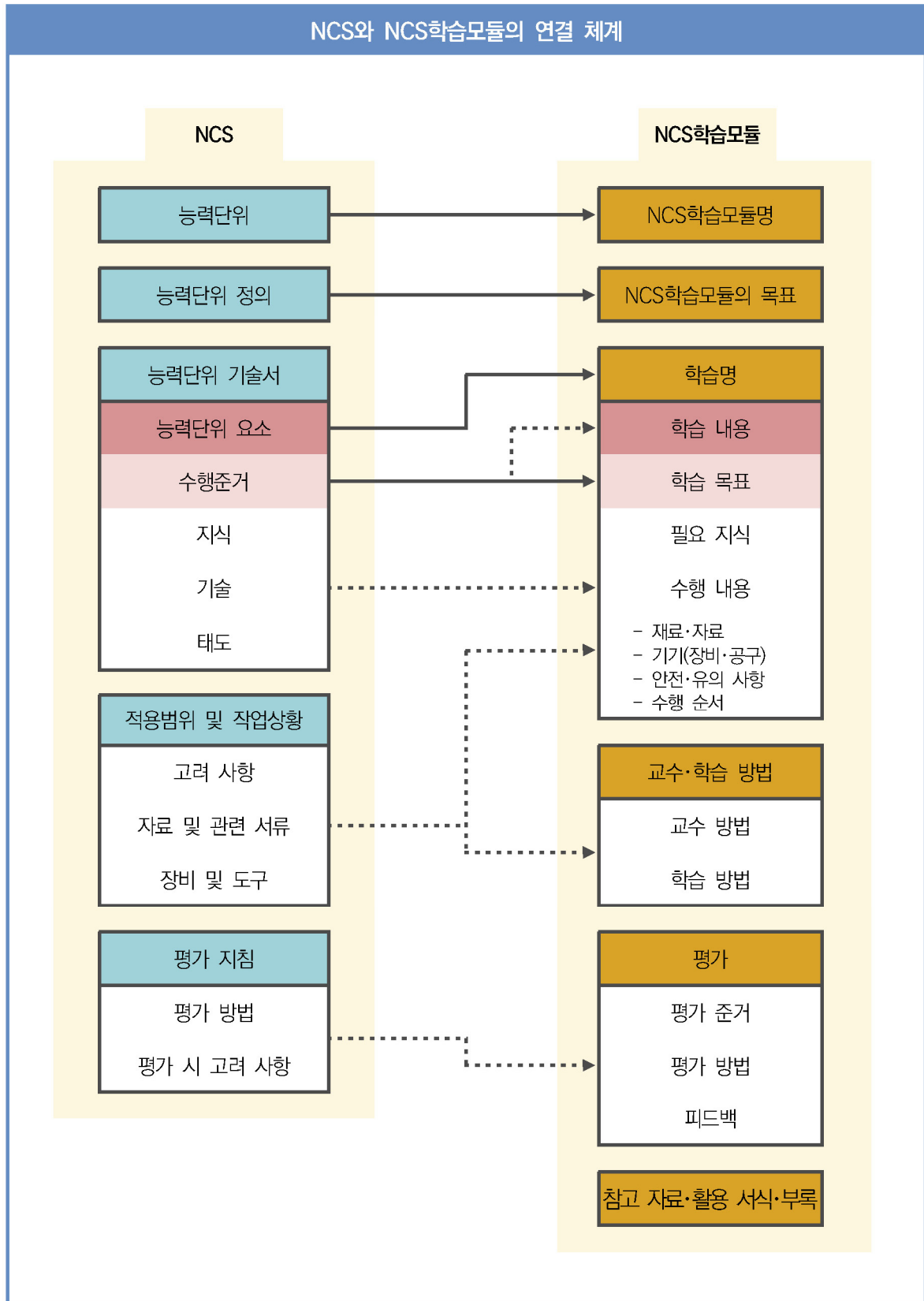
- 국가직무능력표준(NCS: National Competency Standards)이란 산업현장에서 직무를 수행하기 위해 요구되는 지식·기술·소양 등의 내용을 국가가 산업부문별·수준별로 체계화한 것으로 산업현장의 직무를 성공적으로 수행하기 위해 필요한 능력(지식, 기술, 태도)을 국가적 차원에서 표준화한 것을 의미합니다.
- 국가직무능력표준(이하 NCS)이 현장의 ‘직무 요구서’라고 한다면, **NCS학습모듈은 NCS의 능력단위를 교육훈련에서 학습할 수 있도록 구성한 ‘교수·학습 자료’입니다.** NCS학습모듈은 구체적 직무를 학습할 수 있도록 이론 및 실습과 관련된 내용을 상세하게 제시하고 있습니다.



○ NCS학습모듈은 다음과 같은 특징을 가지고 있습니다.

- 첫째, NCS학습모듈은 산업계에서 요구하는 직무능력을 교육훈련 현장에 활용할 수 있도록 성취목표와 학습의 방향을 명확히 제시하는 가이드라인의 역할을 합니다.
- 둘째, NCS학습모듈은 특성화고, 마이스터고, 전문대학, 4년제 대학교의 교육기관 및 훈련기관, 직장교육기관 등에서 표준교재로 활용할 수 있으며 교육과정 개편 시에도 유용하게 참고할 수 있습니다.

○ NCS와 NCS학습모듈 간의 연결 체계를 살펴보면 아래 그림과 같습니다.



II NCS학습모듈의 체계

○ NCS학습모듈은 1. NCS학습모듈의 위치, 2. NCS학습모듈의 개요, 3. NCS학습모듈의 내용 체계, 4. 참고 자료, 5. 활용서식/부록으로 구성되어 있습니다.

1. NCS학습모듈의 위치

○ NCS학습모듈의 위치는 NCS 분류 체계에서 해당 학습모듈이 어디에 위치하는지를 한 눈에 볼 수 있도록 그림으로 제시한 것입니다.

[NCS-학습모듈의 위치]		
대분류	문화·예술·디자인·방송	
중분류	문화콘텐츠	
소분류	문화콘텐츠제작	
세분류		
방송콘텐츠제작	능력단위	학습모듈명
영화콘텐츠제작	프로그램 기획	프로그램 기획
음악콘텐츠제작	아이템 선정	아이템 선정
광고콘텐츠제작	자료 조사	자료 조사
게임콘텐츠제작	프로그램 구성	프로그램 구성
애니메이션 콘텐츠제작	캐스팅	캐스팅
만화콘텐츠제작	제작계획	제작계획
캐릭터제작	방송 미술 준비	방송 미술 준비
스마트문화앱 콘텐츠제작	방송 리허설	방송 리허설
영사	야외촬영	야외촬영
	스튜디오 제작	스튜디오 제작
	...	...

학습모듈은

NCS 능력단위 1개당 1개의 학습모듈 개발을 원칙으로 합니다. 그러나 필요에 따라 고용단위 및 교과단위를 고려하여 능력단위 몇 개를 묶어 1개 학습모듈로 개발할 수 있으며, NCS 능력단위 1개를 여러 개의 학습모듈로 나누어 개발할 수도 있습니다.

2. NCS학습모듈의 개요

○ NCS학습모듈의 개요는 학습모듈이 포함하고 있는 내용을 개략적으로 설명한 것으로

학습모듈의 목표, 선수학습, 학습모듈의 내용 체계, 핵심 용어 로 구성되어 있습니다.

학습모듈의 목표	해당 NCS 능력단위의 정의를 토대로 학습 목표를 작성한 것입니다.
선수학습	해당 학습모듈에 대한 효과적인 교수·학습을 위하여 사전에 이수해야 하는 학습모듈, 학습 내용, 관련 교과목 등을 기술한 것입니다.
학습모듈의 내용 체계	해당 NCS 능력단위요소가 학습모듈에서 구조화된 체계를 제시한 것입니다.
핵심 용어	해당 학습모듈의 학습 내용, 수행 내용, 설비·기자재 등 가운데 핵심적인 용어를 제시한 것입니다.

제작계획 학습모듈의 개요

학습모듈의 목표

본격적인 촬영을 준비하는 단계로서, 촬영 대본을 확정하고 제작 스태프를 조직하며 촬영 장비와 촬영 소품을 준비할 수 있다.

선수학습

제작 준비(LM0803020105_13v1), 섭외 및 제작스태프 구성(LM0803020104_13v1), 촬영 제작(LM0803020106_13v1), 촬영 장비 준비(LM0803040204_13v1.4), 미술 디자인 협의하기(LM0803040203_13v1.4)

학습모듈의 내용체계

학습	학습 내용	NCS 능력단위 요소	
		코드번호	요소 명칭
1. 촬영 대본 확정하기	1-1. 촬영 구성안 검토와 수정	0803020114_16.3.1	촬영 대본 확정하기
2. 제작 스태프 조직하기	2-1. 기술 스태프 조직 2-2. 미술 스태프 조직 2-3. 전문 스태프 조직	0803020114_16.3.2	제작 스태프 조직하기
3. 촬영 장비 계획하기	3-1. 촬영 장비 점검 과 준비	0803020114_16.3.3	촬영 장비 계획하기
4. 촬영 소품 계획하기	4-1. 촬영 소품 목록 작성 4-2. 촬영 소품 제작 의뢰	0803020114_16.3.4	촬영 소품 계획하기

핵심 용어

촬영 구성안, 제작 스태프, 촬영 장비, 촬영 소품

학습모듈의 목표는

학습자가 해당 학습모듈을 통해 성취해야 할 목표를 제시한 것으로, 교수자는 학습자가 학습모듈의 전체적인 내용흐름을 파악하도록 지도할 수 있습니다.

선수학습은

교수자 또는 학습자가 해당 학습모듈을 교수·학습하기 이전에 이수해야 하는 교과목 또는 학습모듈(NCS 능력단위) 등을 표기한 것입니다. 따라서 교수자는 학습자가 개별 학습, 자기 주도 학습, 방과 후 활동 등 다양한 방법을 통해 이수할 수 있도록 지도하는 것을 권장합니다.

핵심 용어는

해당 학습모듈을 대표하는 주요 용어입니다. 학습자가 해당 학습모듈을 통해 학습하고 평가받게 될 주요 내용을 알 수 있습니다. 「NCS 국가직무능력표준」 사이트(www.ncs.go.kr)의 색인(찾아보기) 중 하나로 이용할 수 있습니다.

3. NCS학습모듈의 내용 체계

○ NCS학습모듈의 내용은 크게 **학습**, **학습 내용**, **교수·학습 방법**, **평가**로 구성되어 있습니다.

학습	해당 NCS 능력단위요소 명칭을 사용하여 제시한 것입니다. 학습은 크게 학습 내용, 교수·학습 방법, 평가로 구성되며 해당 NCS 능력단위의 능력단위 요소별 지식, 기술, 태도 등을 토대로 내용을 제시한 것입니다.
학습 내용	학습 내용은 학습 목표, 필요 지식, 수행 내용으로 구성되며, 수행 내용은 재료·자료, 기기(장비·공구), 안전·유의 사항, 수행 순서, 수행 tip으로 구성한 것입니다. 학습모듈의 학습 내용은 실제 산업현장에서 이루어지는 업무활동을 표준화된 프로세스에 기반하여 다양한 방식으로 반영한 것입니다.
교수·학습 방법	학습 목표를 성취하기 위한 교수자와 학습자 간, 학습자와 학습자 간 상호 작용이 활발하게 일어날 수 있도록 교수자의 활동 및 교수 전략, 학습자의 활동을 제시한 것입니다.
평가	평가는 해당 학습모듈의 학습 정도를 확인할 수 있는 평가 준거 및 평가 방법, 평가 결과의 피드백 방법을 제시한 것입니다.

학습 1	촬영 대본 확정하기
학습 2	제작 스태프 조직하기
학습 3	촬영 장비 계획하기
학습 4	촬영 소품 계획하기

2-1. 기술 스태프 조직

학습 목표 • 프로그램 제작에 적합한 기술 스태프를 조직할 수 있다.

필요 지식 /

1. 기술 스태프의 구성
 프로그램의 장르에 따라 구성하는 기술 스태프는 많은 차이가 있다. 같은 장르의 프로그램이라도 그 형식이나 내용, 규모에 따라서 구성되는 기술 스태프의 종류와 인원 수는 천차만별이다.

1. 스튜디오 프로그램
 토크쇼, 종합 구성, 예능과 같은 스튜디오 프로그램은 부조정실과 스튜디오를 사용하여 제작하기 때문에 많은 기술 스태프가 필요하다.

학습은
 해당 NCS 능력단위요소 명칭을 사용하여 제시하였습니다. 하나의 학습은 일반교과의 '대단원'에 해당되며, 학습모듈을 구성하는 가장 큰 단위가 됩니다. 또한 하나의 직무를 수행하기 위한 가장 기본적인 단위로 사용할 수 있습니다.

학습 내용은
 NCS 능력단위요소별 수행준거를 기준으로 제시하였습니다. 일반교과의 '중단원'에 해당합니다.

학습 목표는
 학습 내용을 이수할 때 학습자가 갖춰야 할 행동 수준을 의미합니다. 따라서 수업시간의 과목 목표로 활용할 수 있습니다.

필요 지식은
 해당 NCS의 지식을 토대로 학습에 대한 이해와 성과를 제고하기 위해 반드시 알아야 할 주요 지식을 제시하였습니다. 필요 지식은 수행에 꼭 필요한 핵심 내용을 위주로 제시하여 교수자의 역할이 매우 중요하며, 이후 수행 순서와 연계하여 교수·학습으로 진행할 수 있습니다.

수행 내용 / 기술 스태프 구성표 작성하기

재료·자료

- 방송프로그램 제작 기획서 및 방송 대본, 콘티(continuity), 제작 일정, 운용표
- 장비 및 시설, 제작 시설 배정 의뢰서 및 배정표, 방송 기술 스태프 데이터베이스(DB) 자료

기기(장비·공구)

- 컴퓨터 등

안전·유의 사항

- 프로그램의 내용과 제작 방법을 분석하고, 각 스태프들의 역할을 신중하게 검토한다.

수행 순서

- 1 방송 대본이나 콘티(continuity), 큐 시트를 분석하고, 프로그램의 내용적 특성, 제작 과정에 대한 자료를 수집한다.
 - 2 프로그램 제작 방법을 결정한다.
1. 스튜디오 녹화를 할 것인가, 야외 촬영을 할 것인가 검토한다.

수행 tip

- 스태프의 결정은 스태프 간의 호흡을 중 요시하여 선정해야 프로그램의 질을 향상시킬 수 있다.

수행 내용은

해당 학습모듈에서 제시한 내용 중 기술(skill)을 습득하기 위한 실습과제로 활용할 수 있습니다.

재료·자료는

수행 내용을 수행하는데 필요한 재료 및 준비물로 실습 시 활용할 수 있습니다.

기기(장비·공구)는

수행 내용에 필요한 기본적인 장비 및 도구를 제시하였습니다. 제시된 기기 외에도 수행에 필요한 다양한 도구나 장비를 활용할 수 있습니다.

안전·유의사항은

수행 내용을 수행하는 데 있어 안전상 주의해야 할 점 및 유의사항을 제시하였습니다. 실습 시 유념해야 하며, NCS의 고려 사항도 추가적으로 활용할 수 있습니다.

수행 순서는

실습 과제의 진행 순서로 활용할 수 있습니다.

수행 tip은

수행 내용에서 실습을 용이하게 할 수 있는 아이디어를 제시하였습니다. 수행 tip은 지도상의 안전 및 유의사항 외에 전반적으로 적용되는 주안점 및 수행 과제 목적에 대한 보충설명, 추가사항 등으로 활용할 수 있습니다.

학습2 교수·학습 방법

교수 방법

- 방송 프로그램의 기술적 요소, 미술 구성 요소, 특수 촬영에 대해 설명한다.
- 방송 프로그램 제작에서 각 기술 스태프의 역할에 대해 설명한다.
- 방송 프로그램을 분석하고 필요한 기술 스태프를 구성할 수 있도록 지시한다.

학습 방법

- 방송 프로그램의 기술적 요소, 미술 구성 요소, 특수 촬영에 대해서 알아본다.
- 프로그램 제작에 필요한 기술 스태프의 역할을 이해하고, 기술 스태프 구성표를 작성한다.

교수·학습 방법은

학습 목표를 성취하는 데 필요한 교수 방법과 학습 방법을 제시하였습니다.

교수 방법은

해당 학습 활동에 필요한 학습 내용, 학습 내용과 관련된 자료명, 자료 형태, 수행 내용의 진행 방식 등에 대하여 제시하였습니다. 또한 학습자의 수업참여도 제고 방법 및 수업 진행상 유의사항 등도 제시하였습니다. 선수학습이 필요한 학습을 학습자가 숙지하였는지 교수자가 확인하는 과정으로 활용할 수도 있습니다.

학습 방법은

해당 학습 활동에 필요한 학습자의 자기 주도 학습 방법을 제시하였습니다. 또한 학습자가 숙달해야 할 실기 능력과 학습 과정에서 주의해야 할 사항 등도 제시하였습니다. 학습자가 학습을 이수하기 전 반드시 숙지해야 할 기본 지식을 학습하였는지 스스로 확인하는 과정에 활용할 수 있습니다.

학습2

평가

평가 준거

- 평가자는 학습자가 학습 목표를 성공적으로 달성하였는지를 평가해야 한다.
- 평가자는 다음 사항을 평가해야 한다.

학습 내용	학습 목표	성취수준	상	중	하
기술 스태프 조직	- 프로그램 제작에 적합한 기술 스태프를 조직할 수 있다.				
미술 스태프 조직	- 프로그램 제작에 적합한 미술 스태프를 조직할 수 있다.				
전문 스태프 조직	- 프로그램 특수 촬영을 위한 전문 스태프를 조직할 수 있다.				

평가 방법

- 사례 연구

학습 내용	평가 항목	성취수준	상	중	하
기술 스태프 조직	- 프로그램에서 기술적 요소의 파악 여부 - 기술 스태프의 역할 파악 여부 - 프로그램에 필요한 기술 스태프 구성표 작성 능력				

피드백

- 사례 연구
 - 프로그램을 선택하여 기술 스태프, 미술 스태프, 전문 스태프 구성표를 예시와 같이 작성하였는지 개인별 능력을 평가한 후, 그 결과를 모든 학습자에게 공유하도록 한다.

평가는

NCS 능력단위의 평가 방법과 평가 시 고려사항을 준용하여 작성합니다. 교수자와 학습자가 평가 항목별 성취수준 확인 시 활용할 수 있습니다.

평가 준거는

학습자가 학습을 어느 정도 성취하였는지 평가하기 위한 기준을 제시하고 있습니다. 학습 목표와 연계하여 단위수업 시간에 평가 항목 별 성취수준을 평가하는 데 활용할 수 있습니다.

평가 방법은

NCS 능력단위의 평가 방법을 참고하였으며, 평가 준거에 따른 평가 방법을 2개 이상 제시합니다. 평가 방법의 종류는 포트폴리오, 문제해결 시나리오, 서술형 시험, 논술형 시험, 사례 연구, 평가자 체크리스트, 작업장 평가 등이 있으며, NCS 능력단위 요소 별 수행 수준을 평가하는 데 가장 적절한 방법을 선정하여 활용할 수 있습니다.

피드백은

평가 후에 학습자들에게 평가 결과를 피드백하여 학습 목표를 달성하는 데 활용할 수 있습니다.

4. 참고 자료

참고 자료

- 교육부(2013). 섭외 및 제작스태프 구성(LM0803020104_13v1). 한국직업능력개발원.

참고 자료는

해당 학습모듈에 제시된 인용 자료의 출처를 제시하였습니다. 교수·학습의 과정에서 참고로 활용할 수 있습니다.

5. 활용 서식/부록

활용 서식

스튜디오 기술 스태프 구성표

직종	이름	연락처	소속	특이사항	비고
기술감독					
조명감독					

활용 서식은

평가 서식, 실습 시트 등 교수·학습 시 활용할 수 있는 다양한 서식들로 구성하였습니다. 수행에서 평가에 이르기까지 필요한 서식을 해당 모듈의 특성에 맞춰 개발하거나 기존의 양식을 활용하여 제시하였습니다.

부록

[디지털 텔레비전 방송프로그램 음량 등에 관한 기준]

제정 2014. 11. 29. 미래창조과학부 고시 제2014-87호

제1장 총칙

제1조(목적) 이 고시는 방송법 제70조의2제1항에 따라 방송사업자가 디지털 텔레비전 방송프로그램 및 방송광고의 음량을 일정하게 유지하기 위해 필요한 사항을 규정함을 목적으로 한다.

부록은

활용 서식 이외에 교수·학습 과정에서 참고할 수 있는 자료가 있는 경우 제시하였습니다.

[NCS-학습מוד의 위치]

대분류	정보통신	
중분류	정보기술	
소분류	정보기술개발	

세분류

SW아키텍처	능력단위	학습מוד명
응용SW엔지니어링	요구사항 확인	요구사항 확인
임베디드SW엔지니어링	데이터 입출력 구현	데이터 입출력 구현
DB엔지니어링	통합 구현	통합 구현
NW엔지니어링	정보시스템 이행	정보시스템 이행
보안엔지니어링	제품소프트웨어 패키징	제품소프트웨어 패키징
UI/UX엔지니어링	서버프로그램 구현	서버프로그램 구현
시스템SW엔지니어링	인터페이스 구현	인터페이스 구현
빅데이터플랫폼 구축	애플리케이션 배포	애플리케이션 배포
핀테크엔지니어링	애플리케이션 리팩토링	애플리케이션 리팩토링
데이터아키텍처	인터페이스 설계	인터페이스 설계
IoT시스템연동	애플리케이션 요구사항 분석	애플리케이션 요구사항 분석
인프라스트럭처 아키텍처 구축	기능 모델링	기능 모델링
	애플리케이션 설계	애플리케이션 설계

정적모델 설계	정적모델 설계
동적모델 설계	동적모델 설계
화면 설계	화면 설계
화면 구현	화면 구현
애플리케이션 테스트 관리	애플리케이션 테스트 관리
애플리케이션 테스트 수행	애플리케이션 테스트 수행
소프트웨어공학 활용	소프트웨어공학 활용
소프트웨어개발 방법론 활용	소프트웨어개발 방법론 활용
프로그래밍 언어 응용	프로그래밍 언어 응용
프로그래밍 언어 활용	프로그래밍 언어 활용
응용SW 기초 기술 활용	응용SW 기초 기술 활용
개발자 환경 구축	개발자 환경 구축
개발 환경 운영 지원	개발 환경 운영 지원

차 례

학습모듈의 개요	1
----------	---

학습 1. 구조적 프로그래밍 언어 활용하기

1-1. 구조적 프로그래밍 설계	3
1-2. 구조적 프로그래밍 언어 활용	12
• 교수 · 학습 방법	24
• 평가	25

학습 2. 객체지향 프로그래밍 언어 활용하기

2-1. 객체지향 프로그래밍 설계	27
2-2. 객체지향 프로그래밍 언어 활용	39
• 교수 · 학습 방법	53
• 평가	54

학습 3. 스크립트 언어 활용하기

3-1. 스크립트 언어 설계	56
3-2. 스크립트 언어 활용	63
• 교수 · 학습 방법	71
• 평가	72

참고 자료	74
-------	----

프로그래밍 언어 활용 학습모듈의 개요

학습모듈의 목표

응용소프트웨어 개발에 사용되는 프로그래밍 언어의 기본 문법을 활용하여 기본 응용소프트웨어를 구현할 수 있다.

선수학습

개발자 환경 구축(2001020233_19v4), 프로그래밍 언어 응용(20001020230_19v4), 기능모델링(2001020220_19v2)

학습모듈의 내용체계

학습	학습 내용	NCS 능력단위 요소	
		코드번호	요소 명칭
1. 구조적 프로그래밍 언어 활용하기	1-1. 구조적 프로그래밍 설계	2001020231_19v4.1	구조적 프로그래밍 언어 활용하기
	1-2. 구조적 프로그래밍 언어 활용		
2. 객체지향 프로그래밍 언어 활용하기	2-1. 객체지향 프로그래밍 설계	2001020231_19v4.2	객체지향 프로그래밍 언어 활용하기
	2-2. 객체지향 프로그래밍 언어 활용		
3. 스크립트 언어 활용하기	3-1. 스크립트 언어 설계	2001020231_19v4.3	스크립트 언어 활용하기
	3-2. 스크립트 언어 활용		

핵심 용어

하향식 설계, 절차식 언어, 명령어 언어

학습 1

구조적 프로그래밍 언어 활용하기

학습 2 객체지향 프로그래밍 언어 활용하기

학습 3 스크립트 언어 활용하기

1-1. 구조적 프로그래밍 설계

학습 목표

- 응용소프트웨어 개발을 위하여 프로그램 설계서를 확인할 수 있다.

필요 지식 /

① 구조적 프로그래밍 설계 구조 및 절차

구조적 프로그래밍은 구조화 프로그래밍으로도 불리며 절차적 프로그래밍 기반 아래 만들어진 프로그래밍 기법으로 프로그램을 결합하는 순차, 분기, 반복 구조를 통하여 설계된다.

1. 구조적 프로그래밍 설계 구조

(1) 순차구조

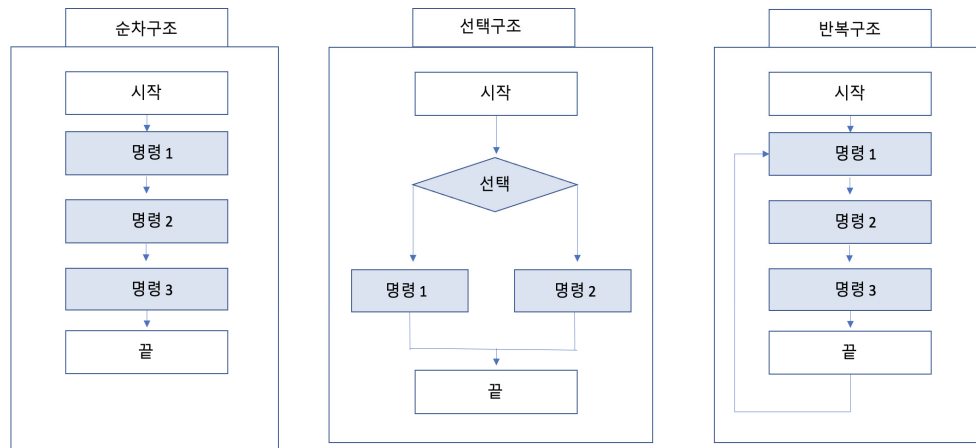
프로그램 코드가 위에서 아래로 일직선으로만 실행되는 구조로 실행을 위하여 작은 단위로 나눈 후 순서대로 진행되는 구조이다.

(2) 선택구조

선택구조는 분기형 구조로서, 주어진 조건 만족 여부에 따라 처리 대상과 순서를 전달하는 구조이다. 어떤 것을 선택하느냐에 따라 실행 순서가 달라지며 조건문(if, if-else)를 이용하여 실행한다. 특정 조건이 들어와 참이냐 거짓이냐에 따라 처리가 분기되는 구조이다.

(3) 반복구조

반복형 구조로 조건이 만족할 때까지 동작을 반복하는 구조이다. 반복문(while, for)을 사용하여 실행한다. 참인 조건이 될 때까지 진행되는 구조이다.



출처: 집필진 제작(2021)
[그림 1-1] 순차구조, 선택구조, 반복구조

2. 구조적 프로그래밍 언어 개발 절차

(1) 요구사항 분석

요구사항 분석 단계는 고객이 원하는 요구사항을 수집하여 명세화하는 단계이다.

(2) 구조적 분석

구조적 분석 단계는 고객이 원하는 기능, 시스템 환경, 데이터를 종합하여 데이터 흐름도(Data Flow Diagram)를 작성하는 단계이다. 시스템이 지원할 비즈니스 프로세스를 하향식(Top-Down)으로 분할하고 시각적으로 표현하는 기법이다. 대표적인 기법에는 ERD(Entity Relation Diagram)과 DFD(Data Flow Diagram)가 있다.

(3) 구조적 설계

구조적 설계 단계는 모듈 중심으로 설계하는 단계로 재활용, 결합도를 낮춰 독립성을 높이는 단계이다.

(4) 구조적 프로그래밍

구조적 프로그래밍 단계는 순차, 선택, 반복의 논리 구조로 구성하여 프로그램 복잡성을 최소화하여 프로그래밍 하는 단계이다.

② 구조적 프로그래밍 설계서 구성요소 및 작성 방법

1. 구조적 프로그램 구성요소

(1) 데이터 흐름도 DFD(Data Flow Diagram)

데이터 각 기능을 분할하여 표현한 구조도이다. 특정 프로세스를 구성하는 하위

프로세스들 사이의 데이터 이동을 보여준다. 최하위 프로세스 수준으로 작성된 DFD는 각 프로그래머가 모듈에 대응하여 개발한다.

(2) 자료사전 DD(Data Dictionary)

자료나 의미나 자료의 단위 및 값에 대한 사항을 정의하는 도구로 DFD에 표현된 자료 저장소를 구체적으로 명시하기 위해 사용하는 도구이다. 프로그램에서 사용하는 모든 데이터 항목의 표준 명칭, 의미, 형식, 길이, 용도, 원천 등을 정의하고 있다. 이를 통하여 서로 다른 모듈에서도 데이터 항목을 일관성 있게 표현하고 식별할 수 있다

(3) 상태 전이도 STD(State Transition Diagram)

보통 어떤 상태에서 다른 상태로 변경되는 과정 및 해당 과정의 프로세스 명세를 작성하는 다이어그램이다.

(4) 소단위 명세 Minispec(Mini Specification)

나눌 수 없는 부분까지 기능을 분리하여 작성한 명세서이다. 구조 언어를 사용하거나 N-S Chart를 이용한다.

2. DFD 기반 구조적 프로그래밍 설계요소

(1) DFD 구성요소

(가) 프로세스

- 1) 프로세스는 원과 원안의 이름으로 표현하고, 데이터를 원하는 데이터로 변환하여 출력시키기 위한 과정이다.
- 2) 원안에 기록하는 이름은 프로세스가 수행하는 프로세스 또는 일을 수행하는 행위자를 기술한다.
- 3) 프로세스는 항상 입력되는 데이터를 입력할 수 있어야 하며, 자체적으로는 데이터를 생성할 수 없다.
- 4) 프로세스는 항상 새롭게 개발할 수 있는 가치를 부가해야 한다.

(나) 데이터 흐름(Data Flow)

- 1) 데이터 흐름은 DFD의 구성요소들 간의 인터페이스를 나타낸다.
- 2) 대부분의 데이터 흐름은 프로세스들 사이를 연결하게 되지만, 데이터 저장소로부터 데이터 흐름을 나타내기도 한다.
- 3) 데이터 흐름은 부여되지 않은 화살표로 표시하거나 명칭을 부여한다. 단, 후속 작업들의 참조를 위해 되도록 명칭을 부여하는 것을 권장한다.
- 4) 서로 다른 데이터 흐름에는 같은 이름을 부여하지 않는다.

(다) 데이터 저장소(Data Store)

- 1) 데이터 저장소는 저장되어 있는 정보 집합이다.

- 2) 데이터 저장소는 저장되는 저장소 파일 등일 수 있으며, 때로는 휴지통일 수도 있다.
- 3) 데이터 저장소는 데이터의 변동을 표시하는 것이 아닌 단순한 데이터 저장을 나타내는 것이다.
- 4) 데이터 흐름을 표시함으로써 데이터의 입출력을 나타낸다.
- 5) 데이터 흐름도에서 데이터 저장소를 나타내는 표기법은 두 개의 직선인 평행선으로 나타내고, 평행선 안에 데이터 저장소의 명칭을 부여한다.

(라) 외부 엔티티(External Entity)

- 1) 액터는 데이터를 생성, 소비함으로써 데이터 흐름도를 주도하는 활성 객체이다. 액터는 데이터 흐름도의 입력과 출력에 붙는다. 데이터 흐름도의 경계에 놓이게 되며, 소스나 싱크로 데이터의 흐름을 중단시킨다.
- 2) 외부 엔티티는 프로세스 처리 과정의 데이터 발생의 시작 및 종료를 나타낸다.
- 3) 외부 엔티티는 데이터 흐름도상에서 프로세스와의 상호관련성을 표시하며, DFD 범위 밖에 사각형 형태로 표시한다.

(2) DFD 설계 방법

- (가) 업무를 분석하여 프로세스에 대한 입출력 데이터 흐름을 식별한다. 이후 업무의 주변 경계에 그들을 표시한다.
- (나) 데이터 흐름에 필요하고 제공되어야 할 외부 엔티티를 정의한다.
- (다) 입력으로부터 출력으로나 출력으로부터 입력으로, 중간 지점부터의 데이터 흐름을 식별한다.
- (라) 모든 접속 관계 데이터 흐름에 명칭을 부여한다.
- (마) 프로세스에 대해 입력 데이터와 출력 데이터 흐름의 명칭에 따라 이름을 부여한다.
- (바) 프로세스에 관련된 데이터 저장소를 정의하고 검토, 보완한다.
- (사) 상위 레벨 DFD 작성 후 다음 하위 레벨의 DFD로 분할하여 최하위 레벨까지 그린다.
- (아) 데이터 흐름도의 규모가 커서 한 장의 종이에 그릴 수 없을 때는 시스템을 서브 시스템들로 분할한다. 분할된 서브시스템들의 규모가 클 때는 다시 분할을 계속한다. 세분화를 계속하여 최종 데이터 흐름도를 단순한 기능들만으로 그릴 수 있는 단계까지 분할한다.

재료 · 자료

- 요구사항 정의서, 구조적 프로그래밍 설계서, 기능 명세 설계서
- 업무 정의서(업무 프로세스, 업무 처리 기준 등)

기기(장비 · 공구)

- 전산 장비(컴퓨터, 프린터, 빔 프로젝터, 네트워크 장비 등)
- OA 도구: 워드 프로세서, 스프레드시트, 프레젠테이션 도구 등
- CASE 도구: HIPO, SREM, PSL/PSA 등

안전 · 유의 사항

- 설계를 지원하는 도구들을 다룰 수 있어야 한다.

수행 순서

① 소프트웨어 개발 요구사항을 확인한다.

설계서를 이해하기 위해서는 사용자 혹은 시스템의 요구사항을 확인하여야 한다.

1. 시스템 요구사항 목록을 확인한다.

사용자 혹은 시스템이 요구하는 요구사항의 목록을 확인한다. 요구사항 목록은 크게 기능 요구사항과 비기능 요구사항으로 이루어져 있다. 일반적으로 시스템 요구사항에는 기능, 보안, 성능 데이터, 테스트, 품질, 제약사항, 프로젝트 관리, 프로젝트 지원 등 다양한 시스템 요구사항이 분류되어 있다. 아래 사례는 기능 모델 다이어그램 작성을 위해 요약한 요구사항 목록의 예이다.

2. 기능 모델링을 위한 기능 요구사항을 확인한다.

학습 수준에서 수행되는 기능 모델링에서는 시스템의 특성에 따른 기능 요구사항을 분석하여 시스템의 요구사항을 분석하는 데 중점을 둔다. 기능 모델링 수행 시 보안, 테스트, 인터페이스 등 비기능 요구사항에 대해서도 고려하여 모델링을 수행하면 사용자의 만족도를 증대시킬 수 있다.

(1) 기능 요구사항 목록 분류를 확인한다.

요구사항 전체 목록 중 기능 요구사항만 분류하여 기능을 확인한다.

〈표 1-1〉 근태 관리 시스템의 요구사항 목록 사례

요구사항 분류	요구사항 고유 번호	요구사항 명칭
분석/설계 - APR (Analysis Plan Requirement)	APR-001	요구사항 수집 및 확정
	APR-002	프로세스 분석
	APR-003	화면 및 DB 설계
기능 - SFR (System Function Requirement)	SFR-001	출퇴근 현황 조회
	SFR-002	관리자 기능
	SFR-003	외출 신청 기능
	SFR-004	유연 근무제 관리 기능
	SFR-005	특근 신청 관리 기능
	SFR-006	최종 점검 관리 기능
	SFR-007	초과 근무 관리 기능
	SFR-008	권한 관리 기능
성능 - PER (Performance Requirement)	PER-001	기존 시스템 성능 유지
인터페이스 - SIR (System Interface Requirement)	SIR-001	시스템 연계
	SIR-002	직관적 UI

출처: 교육부(2017). 기능모델링(LM2001020220_19v2). 한국직업능력개발원, p.21.

〈표 1-2〉 근태 관리 시스템의 기능 요구사항 분류 사례

요구사항 분류	요구사항 고유 번호	요구사항 명칭
기능 - SFR (System Function Requirement)	SFR-001	출퇴근 현황 조회
	SFR-002	관리자 기능
	SFR-003	외출 신청 기능
	SFR-004	유연 근무제 관리 기능
	SFR-005	특근 신청 관리 기능
	SFR-006	최종 점검 관리 기능
	SFR-007	초과 근무 관리 기능
	SFR-008	권한 관리 기능

출처: 교육부(2017). 기능모델링(LM2001020220_19v2). 한국직업능력개발원, p.22.

(가) 기능 요구사항 상세를 확인한다.

〈표 1-3〉 근태 관리 시스템의 기능 요구사항 상세 사례 - 출퇴근 현황 조회

요구사항 고유 번호		SFR-001
요구사항 명칭		출퇴근 현황 조회
요구사항 분류		근태 현황 관리
정의		출퇴근 현황 조회
요구사항 상세 설명		○ 개인 본인만 볼 수 있는 출퇴근 조회 화면 추가
	세부 내용	- 출퇴근 누락 및 오류 시 누락분 관리자 신청 처리 - 출퇴근 및 외출, 유연 근무 내역, 초과 근무 시간 수 등 다양한 정보 확인 기능
		○ 출퇴근 연계 데이터는 보안업체로부터 수신받은 내역을 DB 서버에 일일 전송된 내역을 가지고 처리
요구사항 고유 번호		SFR-002
요구사항 명칭		관리자 기능
요구사항 분류		근태 현황 관리
정의		출퇴근 현황 조회
요구사항 상세 설명		○ 개인 본인만 볼 수 있는 출퇴근 조회 화면 추가
	세부 내용	- 출퇴근 누락 및 오류 시 누락분 관리자 신청 처리 - 출퇴근 및 외출, 유연 근무 내역, 초과 근무 시간 수 등 다양한 정보 확인 기능
		○ 출퇴근 연계 데이터는 보안업체로부터 수신받은 내역을 DB 서버에 일일 전송된 내역을 가지고 처리

출처: 교육부(2017). 기능모델링(LM2001020220_19v2). 한국직업능력개발원. p.22.

요구사항 목록에 정리된 각 요구사항마다 요구사항 상세라는 항목으로 설명과 정의가 달려 있다. 이 부분을 이해한다.

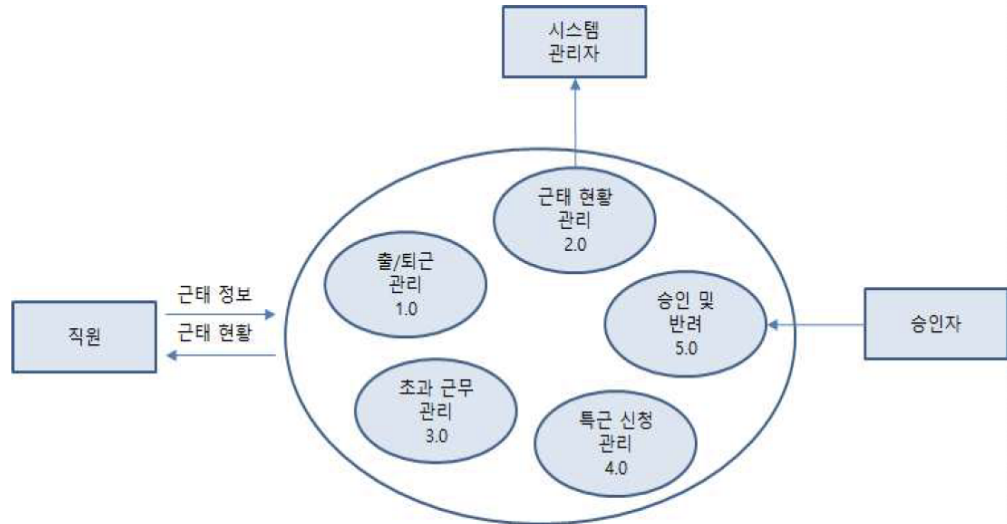
② 설계된 다이어그램 설계서를 확인한다.

디마르코(DeMarco)가 고안한 구조적 분석 기법은 전통적인 데이터 처리 시스템을 분석하고 개발하는 데 사용되고 있다.

1. 배경도(Context Diagram)를 이해한다.

배경도는 요구사항 목록에 기술되어 있는 데이터와 시스템이 반응해야 하는 행위 및

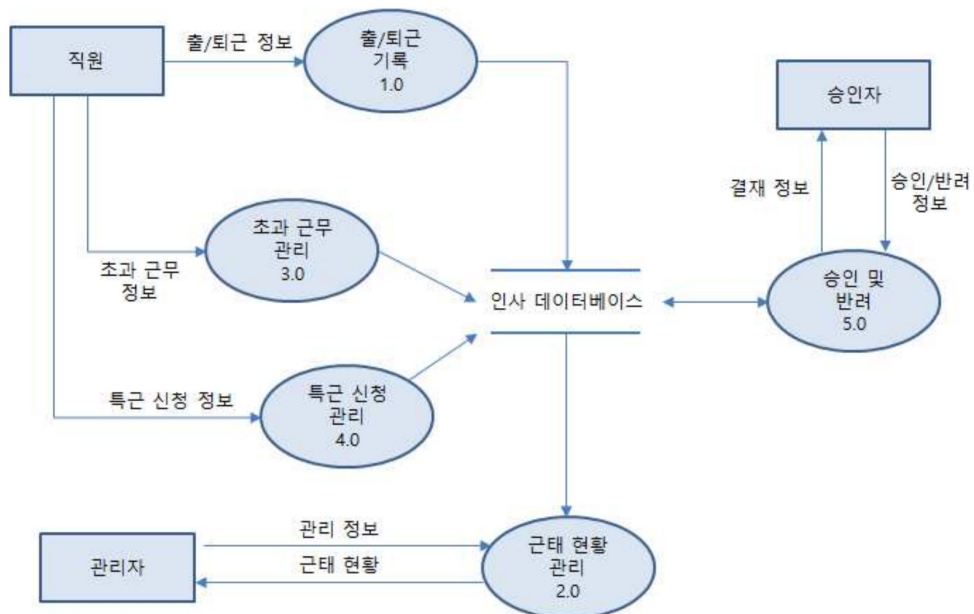
사건 등에서 시스템과 외부 객체 사이에 오가는 정보를 추출하여 설계되어 있다. 배경도를 통해 전체 기능을 파악한다.



출처: 교육부(2017). 기능모델링(LM2001020220_19v2). 한국직업능력개발원. p.23.
[그림 1-2] 근태 관리 시스템의 배경도 예시

2. 자료 흐름도를 확인한다.

프로세스마다 레벨이라는 번호가 부여되어 있으며, 프로세스를 찾아내면서 자연스럽게 자료 저장소의 위치를 확인한다.



출처: 교육부(2017). 기능모델링(LM2001020220_19v2). 한국직업능력개발원. p.24.
[그림 1-3] 근태 관리 시스템의 레벨(Level) 0 자료 흐름도 예시

3. 자료 사전(Data Dictionary)을 확인한다.

자료 사전은 자료 흐름도에 등장한 데이터에 관한 정보를 한곳에 모아놓는 저장소로 개발자나 사용자, 즉 이해관계자(Stakeholder)들이 편리하게 사용할 수 있게 해준다.

수행 tip

- 소프트웨어 요구사항을 명확히 파악해야 구조적으로 프로그램을 이해할 수 있다.
- 개발을 하기 위해서는 설계서의 내용과 함께 시스템 환경 및 성능 등이 고려되어야 한다.

1-2. 구조적 프로그래밍 언어 활용

학습 목표

- 구조적 프로그래밍 언어를 활용하여 애플리케이션을 작성할 수 있다.
- 작성된 애플리케이션의 오류를 식별하고 수정할 수 있다.

필요 지식 /

① 구조적 프로그래밍 언어 개요

구조적 프로그래밍은 구조화된 조직 체계의 모양처럼 하나의 프로그램이 구조를 갖도록 만들어 나가는 프로그래밍 방법을 말한다. 기본 제어 구조인 순차 구조, 선택 구조, 반복 구조를 이용하여 프로그램의 흐름을 구조화시키고 간결하게 만드는 방식으로 사용하는 언어이다.

1. 절차식 언어

구조적 프로그래밍은 프로그램이 실행될 때 위에서 아래로 순서대로 실행되는 방식의 프로그램이다. 구조적 프로그래밍의 이전의 프로그래밍에서는 Goto문을 사용함으로써 진행 방향이 일괄적이지 않았던 단점을 개선한 언어이다.

2. 명령어 언어

프로그램은 순차적으로 실행되면서 필요한 함수를 호출하거나, 실행을 시킬 수 있는데 명령어들을 이용하여 진행되어 진다. 명령은 메인에서 서브프로그램으로 전달 되는데 하향식 방식을 이용한다.

3. 함수 중심 언어

구조화를 시켜가면서 독립적인 행위를 할 수 있는 작은 단위로 나뉘는데, 이때 함수를 이용하여 구성하게 된다. 이런 함수는 서브프로그램이 될 수 있다.

② 구조적 프로그래밍 유형

1. C 언어

C 언어는 미국의 벨 연구소에서 개발한 시스템 소프트웨어 개발 언어로 향후 일반 용도로도 널리 사용되고 있다. C 언어는 이식성이 좋고, 구조화 프로그램을 작성하기가 용이하다. 고급언어와 저급언어의 특성을 동시에 가짐으로써 일반 응용소프트웨어에서 시스템의 제어나 자료의 처리를 담당하는 소프트웨어에 사용된다.

2. 파스칼(Pascal)

파스칼은 구조적 프로그래밍의 개념을 살려 니콜라우스 워드가 개발한 언어이다. 알골에서 명시적 변수 선언, 블록 구조, 제어 구조문 등의 특징을 발췌하여 자료 구조의 기능을 보강한 언어이다. 또한 문법이 간결하면서도 사용하기가 편리하고 프로그램의 이해가 쉬워 구조적 프로그램 언어로도 널리 이용되고 있다.

3. 에이다(Ada)

에이다 언어는 1970년대 초에 개발되었으나 파스칼 언어로부터 에이다 언어는 많은 영향을 받았다. 높은 신뢰도를 갖고 장시간 운용 가능한(Long-Lived) 대규모 시스템(System in the Large)을 구성하기 위한 컴퓨터 프로그래밍 언어이다.

③ 주요 구현 언어 문법

1. 조건문

명령어 흐름을 제어할 수 있는 역할을 하는 명령어 문장을 조건문이라고 한다.

(1) if문

```
if (조건문)
    문장 1;
    문장 2
```

if 조건문에 따라 문장을 분기하며 조건문의 결과는 참(TRUE, 1) 또는 거짓(FALSE, 0)이 나온다. 참이면 문장 1을 수행한 후에 문장 2를 수행하고, 거짓이면 문장 1을 수행하지 않고 바로 문장 2를 수행한다. 조건문의 결과에 따라 문장 1이 수행되거나 수행되지 않는다. if 조건문이 참이면 문장 1을 수행하고 문장 2로 제어가 넘어가고, 거짓이면 문장 1을 수행하지 않고 바로 다음 문장 2로 제어가 넘어간다.

(2) if~else문

if문은 한 조건에 대하여 확인을 할 수 있고, 다중의 조건을 체크하기 위해 if~else 문을 이용할 수 있다. 조건문이 참이면 A를 출력하고, 그렇지 않으면 B를 출력하는 조건문을 만들 수 있다.

```

main() {
    int a = 5;

    if (a > 3)                // if (참)
        printf("A\n");      // 조건문이 참이면 실행
    else
        printf("B\n");      // 조건문이 거짓이면 실행
    printf("C\n");

    if (a > 8) printf("D\n"); // 조건문이 참이면 실행

    else printf("E\n");      // 조건문이 거짓이면 실행

    printf("F\n");
}

```

(3) switch 분기문

switch 구문 옆에 있는 수식 또는 변수의 결과는 반드시 -1, 0, 1, 2 같은 정수 값이어야 한다. 수식 또는 변수의 결과인 정수값이 나오면 그 값에 해당하는 case 값을 찾아 해당하는 명령어를 수행하고 switch 구문을 종료한다.

만약 수식 또는 변수의 결과에 해당하는 정수값을 갖는 case 구문이 없을 때는 default 구문을 수행하고 switch 구문을 종료한다. default 구문은 프로그래밍을 해도 되고 하지 않아도 된다. 그런데 만약 case 값 중에 수식 또는 변수 결과의 정수값이 없고 default 구문도 없다면, 전체 case 조건문에서는 어떠한 구문도 수행하지 않고 종료하게 된다.

```

switch (수식 또는 변수) {
case 값:
    A;
case 값:
    B;
case 값:
    C;
default:
    D;
}

```


2. 반복문

(1) for 반복문

```
for (A; B; C)
  A: 초깃값
  B: 반복 조건식
  C: 증가 또는 감소
```

for 반복문은 하나 이상의 수행하고자 하는 문장 또는 명령어를 정해진 횟수만큼 반복적으로 수행하는 일을 한다. for 반복문에서 A 항목은 반복하고자 하는 횟수의 초깃값을 의미한다. 초깃값은 말 그대로 처음에 한 번 지정하는 값으로, 반복문을 수행하는 동안 C 항목(증가식 또는 감소식)에 따라 변한다.

(2) while 반복문

while 반복문은 for 반복문과 마찬가지로 반복적으로 일을 수행하고자 할 때 사용하는 문장이다. for 반복문과 while 반복문은 수행하는 일은 같지만 차이점이 있다. for 반복문은 “10번 반복하세요” 같은 형태이고, while 반복문은 “10이 될 때까지 반복하세요” 같은 형태로 구분할 수 있다.

〈표 1-4〉 for 문, while 문 예시

for 반복문	while 반복문
for (i = 1 ; i < 11; i++) 문장;	while (i < 11) 문장;

(3) do-while 반복문

do-while 반복문은 while과 같이 반복적으로 일을 수행할 때 사용하는 반복문이다. do-while 반복문은 반복적으로 수행하는 문장 및 문장들을 최소 한 번은 수행하고, 그 이후에 반복문 조건을 검사하여 참일 때만 반복적으로 수행한다는 점에서 while 반복문과 다르다.

재료 · 자료

- 소프트웨어 개발 요구사항
- 소프트웨어 상세 설계서
- 소프트웨어 개발 가이드
- 개발 공통 코드 정의서

기기(장비 · 공구)

- 컴퓨터, 네트워크 장비, 프린터, 빔 프로젝트
- CASE 도구(SW 개발 지원 도구, SW 테스트 지원 도구 등)
- 형상관리 도구

안전 · 유의 사항

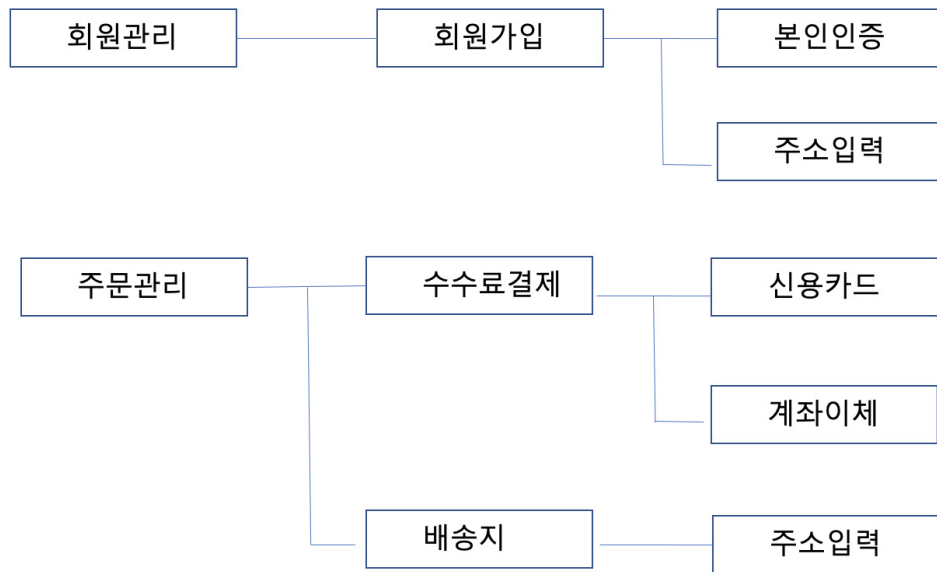
- 프로그램의 요구사항과 환경을 명확히 파악한다.
- 프로그램의 문법을 명확히 이해한다.
- 구조적 프로그래밍 설계 방법에 대하여 이해한다.

수행 순서

① 프로그램 기능을 도출한다.

1. 요구사항과 설계서를 분석하여 기능을 도출한다.

- (1) 구현해야 할 기능을 도출하여 관련 기능이 누락되지 않았는지 확인하고, 구현해야 할 프로그램 범위의 산정과 계획을 수립한다.
- (2) 특정 기능 중심으로 구현할 수 있도록 모듈을 정한다.
- (3) 전체 개발해야 할 범위에 대해 고민한다.
- (4) 메뉴 구조도와 같이 계층화시켜 분석하면 용이하다.



출처: 집필진 제작(2021)
[그림 1-4] 프로그램 기능 예시

2. 기능적 모듈화를 고려한다.

모듈과 컴포넌트를 이용하여 기능적으로 분리하여 효율적인 프로그래밍이 될 수 있도록 개발한다.

(1) 응집도를 고려한다.

응집도는 모듈 내부 요소들이 서로 연관되어 있는 정도를 나타내는 정도이다. 응집도가 강할수록 품질이 높고, 약할수록 품질이 낮다.

응집도는 기능적, 순차적, 교환적, 절차적, 시간적, 논리적, 우연적 응집도로 7가지의 응집도가 있다.

(가) 응집도를 높힐 수 있도록 요구사항 단계에서부터 모듈의 이해가 필요하다.

(나) 응집도를 높게 개발하면 유지보수할 때 도움을 줄 수 있다.

(다) 응집도 유형 세부적으로 하나씩 대입해서 고민하기보다는 응집도를 높힐 수 있도록 고민하여 개발한다.

〈표 1-5〉 응집도 사례

응집도	내용
기능적 응집도	한 모듈을 구성하는 요소들이 단일 기능을 수행하기 위해 협력하는 응집도
순차적 응집도	모듈을 구성하는 한 요소의 출력이 다음 요소의 입력 데이터로 사용하는 경우의 응집도
교환적 응집도	동일한 입력과 출력을 사용하여 서로 다른 기능을 수행하는 구성요소들이 모였을 경우의 응집도
절차적 응집도	모듈이 가지는 기능들이 순서에 의해서 처리되는 경우의 응집도
시간적 응집도	특정 시간에 처리되는 기능을 모아 하나의 모듈로 작성할 경우의 응집도
논리적 응집도	모듈의 구성요소들이 유사한 기능을 수행하는 경우의 응집도
우연적 응집도	연관성이 없는 요소로만 구성되는 경우의 응집도

(2) 결합도를 고려한다.

결합도는 모듈 간에 상호 의존하는 정도, 두 모듈 사이의 연관 관계를 말하고 결합도가 약할수록 품질이 높고, 강할수록 품질이 낮다.

〈표 1-6〉 결합도 사례

결합도	내용
내용결합도	두 모듈이 코드를 공유하는 경우 내부 기능이나 그 내부 자료를 직접 참조하거나 수정하는 경우의 결합도
공통결합도	두 모듈이 전역변수를 공유하는 경우의 결합도
외부결합도	소프트웨어 외부의 환경에 좌우되는 경우의 결합도
제어결합도	한 모듈의 데이터가 다른 모듈의 실행 순서를 결정하기 위해 전달되는 경우의 결합도
스텝결합도	배열, 레코드, 구조체 등의 자료구조가 전달될 때의 결합도.
자료결합도	모듈 간의 인터페이스 자료 요소로만 구성될때의 결합도.

② 구조적 프로그래밍 애플리케이션을 작성한다.

1. 변수를 선언한다.

(1) 전역변수 및 지역변수를 선언한다.

변수는 선언 위치에 따라 변수를 접근하는 범위가 달라진다. 함수 밖에서 선언하는 전역변수(Global Variable)와 지역변수(Local Variable)를 선언한다.

(가) 전역변수

전역변수로 선언하면 그 변수는 현재 우리가 프로그래밍하고 있는 C 프로그램 안의 모든 함수에서 자유롭게 접근하여 사용할 수 있다. 프로그램이 종료될 때까지 메모리를 반환하지 않기 때문에 변수를 사용할 수 있다.

(나) 지역변수

지역변수는 작성하고 있는 함수 지역에서 생성하고 바로 자신이 생성한 함수 안에서만 사용하는 변수이다. 함수가 완료되면 사라진다.

(2) 기능적 구현을 한다.

다음과 같은 항목을 고려하여 기능적 구현을 할 수 있다.

(가) 간단한 시퀀스를 구현할 수 있다.

(나) 조건이나 대안을 고민하여 구현할 수 있다.

(다) 반복 및 주기와 반복을 이용하여 구현할 수 있다.

2. 함수를 적용하여 구현한다.

함수를 적용하면 특정한 기능을 독립적으로 수행하도록 모듈화된 프로그램을 구현할 수 있다.

(1) 함수의 선언을 한다.

함수선언을 통해서 함수 이름, 함수의 입력과 출력이 무엇인지 명시한다. 함수를 선언할 때는 출력 자료형과 함수 이름을 먼저 쓰고 괄호() 안에 함수에 들어오는 입력 변수의 자료형과 변수 이름을 쓴다. 그리고 세미콜론(;)으로 함수선언을 마무리한다.

(2) 함수 구성을 한다.

함수선언은 출력 데이터형이 먼저 나오고, 그 다음 함수 이름이 나온 후 괄호 안에 인자형이 들어간다.

출력 데이터형	함수 이름	(입력 자료형과 변수, 입력 자료형과 변수, ...);
---------	-------	--------------------------------

함수 적용 전

```
#include <stdio.h>

main()
{
    int a = 1, b = 3, c = 5;

    a = a * 10;
    printf("10배가 된 값은 %d입니다.\n", a);

    b = b * 10;
    printf("10배가 된 값은 %d입니다.\n", b);

    c = c * 10;
    printf("10배가 된 값은 %d입니다.\n", c);
}
```

함수 적용 후

```
#include <stdio.h>

func10(int x); // 함수선언

main()
{
    int a = 1, b = 3, c = 5;

    func10(a);
    func10(b);
    func10(c);
}

func10(int x) // 함수 몸체 { x = x * 10; printf("10배가 된 값은 %d입니다\n", x); }
```

(3) 함수 호출을 한다.

함수는 호출되면 해당 함수가 호출되고, 인자를 받아 처리하고 다시 호출한 부분으로 돌아가 다음 함수를 처리하게 된다. 다음은 최대값을 구하는 기능을 구현한 함수의 예시이다. 함수명은 `get_max()`이다.

함수 호출

```
#include <stdio.h>

int get_max(int x, int y);

int main()
{
    int a, b;
    int max;

    printf("숫자 두개를 입력하세요 : ");
    scanf("%d %d", &a, &b);

    max = get_max(a, b);

    printf("둘 중 큰 수는 %d.\n", max);

    return 0;
}

int get_max(int x, int y)
{
    return (x > y ? x : y);
}
```

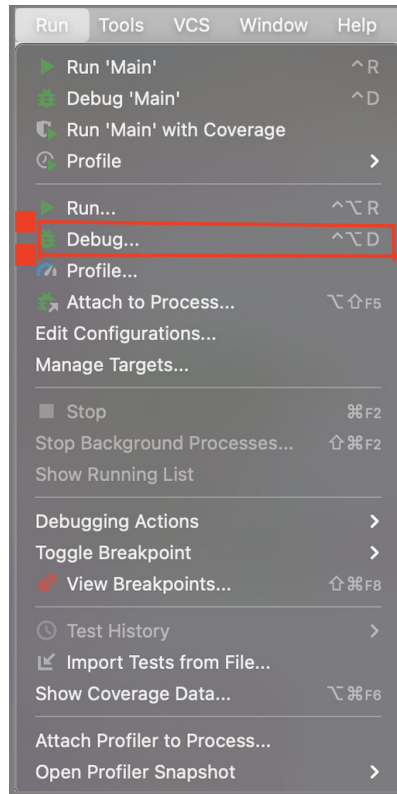
③ 구조적 프로그래밍 애플리케이션 오류 처리를 한다.

1. 프로그램을 실행한다.

개발된 단위 프로그램을 실행한다.

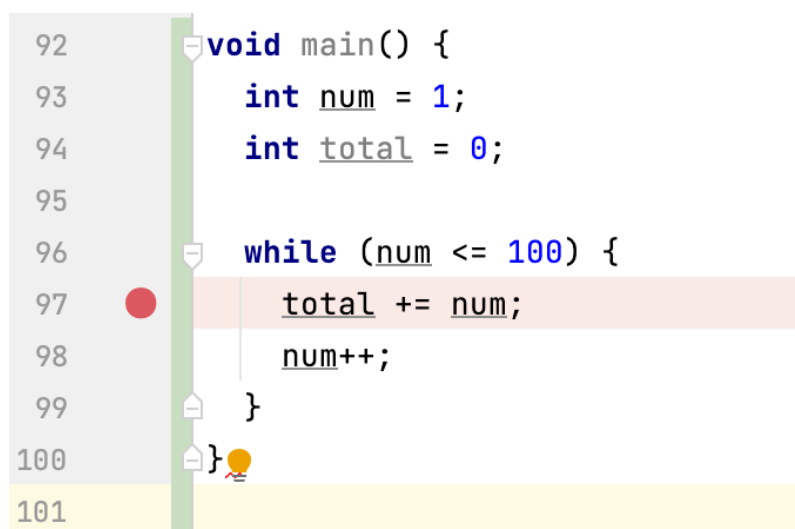
2. 디버깅을 실행한다.

(1) 디버깅 시작을 누른다.



출처: 집필진 제작(2021)
[그림 1-5] 디버깅 호출

- (2) 프로그램을 실행하다가 중단점을 만나면 멈춘다.
- (가) 실행 도중 설정해 놓은 중단점이 있다면 멈춘다.
 - (나) 중단점에서 해당 변수나 로직을 확인한다.



출처: 집필진 제작(2021)
[그림 1-6] 중단점 설정

3. 한 줄씩 실행하면서 변수값을 확인하거나 실행된 화면을 보면서 오류를 확인한다.
4. 디버깅을 종료하고 오류를 수정한다.

수행 tip

- 오픈소스 라이선스에 유의하며 최적화된 도구를 선택하여 진행한다.
- 프로그래밍 문법을 정확히 알고 있어야 한다.

교수 방법

- 제안 요청서, 제안서, 요구사항 정의서 등 요구사항 분석 산출물을 사전에 준비하여 실무적인 내용으로 설명한다.
- 구조적 프로그램의 이해를 높이기 위해 실무 중심의 프로젝트를 선정하고, 요구사항 분석 산출물을 기반으로 구현 내용에 대하여 설명한다.
- 설계서에서 기능, 비기능 요구사항을 식별하는 방법을 설명하고, 기능 모델링 시 비기능 요구사항의 내용을 고려하면서 설명한다.
- 최적화된 코드로 작성되어야 함을 인지시키며, 효율적 코드 작성의 필요성을 제시하고 해당 내용을 설명한다.
- 함수의 기능을 비롯하여 구조적 프로그래밍의 문법에 대하여 설명한다.
- 디버깅 기능의 필요성과 활용 방법에 대하여 설명한다.

학습 방법

- 구조적 프로그래밍 문법을 학습한다.
- 소프트웨어 요구사항에 맞게 구조적 프로그래밍 설계를 한다.
- 설계 산출물에서 기능, 비기능 요구사항을 식별하는 방법을 이해하고, 기능 모델링 시 비기능 요구사항의 내용을 고려하여 학습한다.
- 구조적 프로그래밍에서 필요한 설계서를 이해하고, 모듈화할 수 있는 방법을 연구한다.
- 단위 테스트 및 디버깅하는 방법을 학습한다.

평가 준거

- 평가자는 학습자가 학습 목표를 성공적으로 달성하였는지를 평가해야 한다.
- 평가자는 다음 사항을 평가해야 한다.

학습 내용	학습 목표	성취수준		
		상	중	하
구조적 프로그래밍 설계	- 응용소프트웨어 개발을 위하여 프로그램 설계서를 확인할 수 있다.			
구조적 프로그래밍 언어 활용	- 구조적 프로그래밍 언어를 활용하여 애플리케이션을 작성할 수 있다.			
	- 작성된 애플리케이션의 오류를 식별하고 수정할 수 있다.			

평가 방법

- 평가자 체크리스트

학습 내용	평가 항목	성취수준		
		상	중	하
구조적 프로그래밍 설계	- 응용소프트웨어 개발을 위한 프로그램 설계서 확인 능력			
	- 데이터 흐름도 설계 파악 능력			
구조적 프로그래밍 언어 활용	- 구조적 프로그래밍 언어를 활용한 애플리케이션 작성 능력			
	- 작성된 애플리케이션의 오류 식별 및 수정 능력			

• 포트폴리오

학습 내용	평가 항목	성취수준		
		상	중	하
구조적 프로그래밍 설계	- 응용소프트웨어 개발을 위한 프로그램 설계서 확인 능력			
	- 데이터 흐름도 기반 기능 도출 능력			
구조적 프로그래밍 언어 활용	- 구조적 프로그래밍 언어를 활용한 애플리케이션 작성 능력			
	- 작성된 애플리케이션 오류 식별 및 수정 능력			

피드백

1. 평가자 체크리스트

- 실습 과정을 체크리스트에 따라 평가한 후에 개선해야 할 사항 등을 정리하여 학습자에게 설명한다.
- 설계서를 분석하여 작성된 개발 모듈이 설계서대로 개발되었는지 확인하여 비정상 동작 시 그 문제점을 정리하여 개선 사항을 제시한다.
- 기능 모듈화에 대한 부족한 이해도를 보충할 수 있도록 실무 자료를 제시한다.
- 다양한 데이터 흐름도 사례를 보여주면서 설계 이해 경험을 제시한다.

2. 포트폴리오

- 제출한 보고서를 평가한 후에 주요 사항에 대하여 의견을 제시한다.
- 개발 언어와 도구의 활용을 통하여 정상적으로 동작하는지를 보고 비정상 동작의 원인과 조치 방안에 대해 설명한다.
- 개발된 공통 모듈의 결합도와 응집도의 이해도를 확인하여 미비 사항을 설명한다.
- 개발 언어 문법의 오류를 확인하여 개선사항을 설명한다.
- 디버깅 방법에 대해 정확히 이해하고 있는지 확인하여 디버깅 방법을 명확하게 설명한다.

학습 1	구조적 프로그래밍 언어 활용하기
학습 2	객체지향 프로그래밍 언어 활용하기
학습 3	스크립트 언어 활용하기

2-1. 객체지향 프로그래밍 설계

학습 목표 • 응용소프트웨어 개발을 위하여 프로그램 설계서를 확인할 수 있다.

필요 지식 /

① 객체지향 설계 원칙의 이해

1. 객체지향 속성

(1) 캡슐화

캡슐화는 객체의 속성과 행위를 하나로 묶고, 실제 구현 내용 일부를 외부에 감추어 은닉하는 기법이다. 외부 객체는 객체 내부의 구조를 알지 못하게 하고, 내부 객체에서 제어하여 제공하는 필드와 메소드만 이용할 수 있다.

(2) 추상화

추상화는 어떤 실체로부터 공통적인 부분이나 관심 있는 특성들만 하나로 모은 것을 의미한다. 어떤 하위 클래스들에 존재하는 공통적인 메소드를 인터페이스로 정의하는 것도 이에 포함된다.

(3) 다형성

다형성은 같은 모양의 함수가 상황에 따라 다르게 동작하는 것을 의미한다. 다형성을 이용하는 기법은 오버로딩과 오버라이딩이 있다.

(가) 오버로딩

함수 이름은 같으나 함수의 매개 변수 숫자, 타입 등을 다르게 해서 사용하는 기법이다.

(나) 오버라이딩

상위 클래스의 메소드를 하위 클래스에서 똑같은 이름으로 재정의하는 것을 말한다.

(4) 정보 은닉

캡슐화 정보들을 밖에서 접근 불가능하도록 하는 것을 은닉화라고 한다. 프로퍼티(변수)와 메소드(함수)가 하나의 캡슐 안에 묶인다.

(5) 상속성

부모 클래스에서 새로운 기능을 추가해 자식 클래스를 만들어 사용하는 기법을 상속성이라고 한다. 자식 클래스에서 따로 정의하지 않아도 부모 클래스에서 정의된 것들을 자동으로 상속받아 구현할 수 있다.

2. 객체지향 설계 원칙

(1) 단일 책임 원칙 (Single Responsibility Principle)

모든 클래스는 각각 하나의 책임만 가져야 한다. 클래스는 그 책임을 완전히 캡슐화해야 함을 말한다.

(2) 개방-폐쇄 원칙 (Open Closed Principle)

확장에는 열려 있고 수정에는 닫혀 있어 기존의 코드를 변경하지 않고(Closed), 기능을 추가할 수 있도록(Open) 설계가 되어야 한다는 원칙을 말한다.

(3) 리스코프 치환 원칙 (Liskov Substitution Principle)

자식 클래스는 자신의 부모 클래스를 대체할 수 있다는 원칙이다. 부모 클래스가 들어갈 자리에 자식 클래스를 넣어도 잘 작동해야 한다. 자식 클래스는 부모 클래스의 책임을 무시하거나 재정의하지 않고 확장만 수행하도록 해야 만족한다.

(4) 인터페이스 분리 원칙 (Interface Segregation Principle)

한 클래스는 자신이 사용하지 않는 인터페이스는 구현하지 말아야 한다는 원칙이다. 하나의 평범한 인터페이스보다 여러 개의 구체적인 인터페이스가 좋다.

(5) 의존 역전 원칙 (Dependency Inversion Principle)

의존 역전 원칙은 변화가 없는 것에 의존해야 한다는 원칙이다. 구체적인 클래스보다 인터페이스나 추상 클래스와 관계를 맺어야 하는 원칙이다.

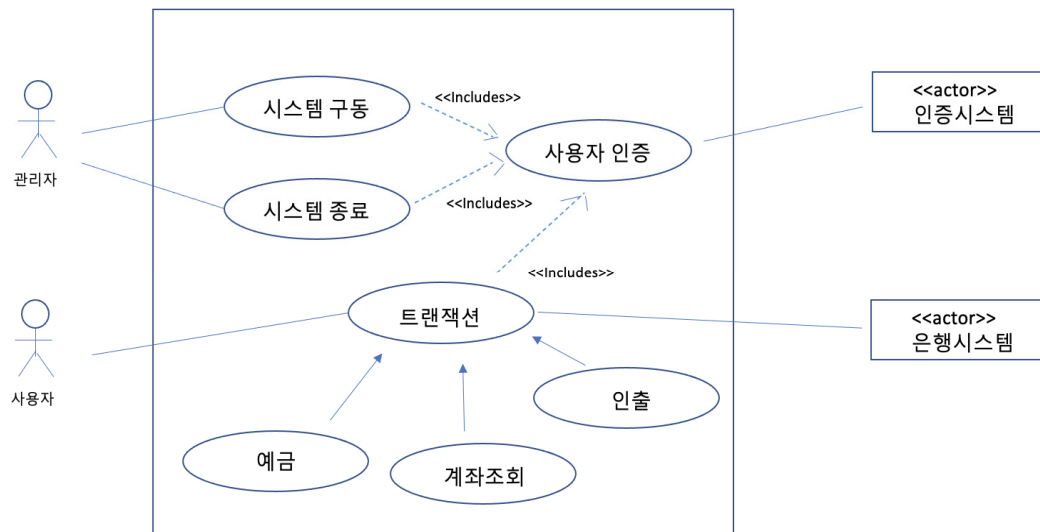
② UML의 이해

1. 유즈케이스 다이어그램

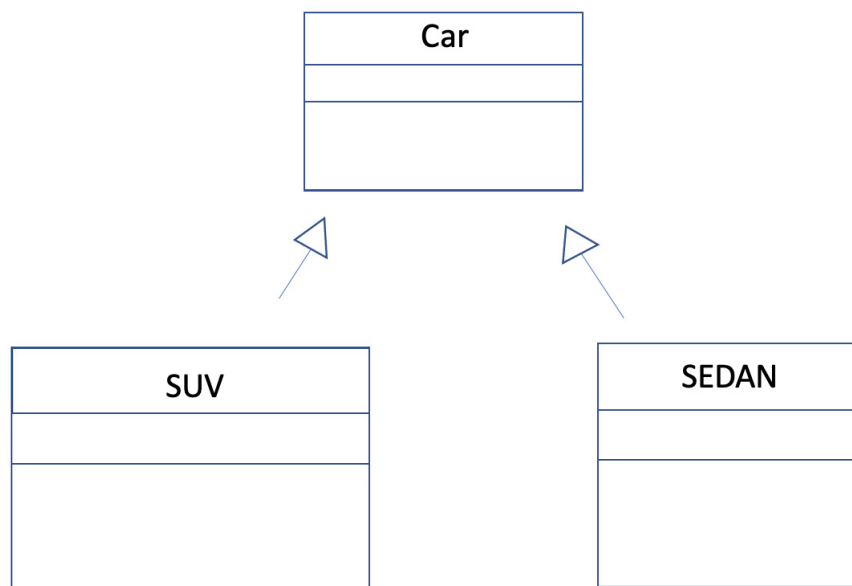
요구 분석, 시스템 설계, 시스템 구현 등의 시스템 개발 과정에서, 개발자 간의 의사소통이 원활하게 이루어지도록 표준화한 대표적인 객체지향 모델링 언어이다.

2. 클래스 다이어그램(Class Diagram)

클래스 다이어그램은 UML의 구조 다이어그램으로서 클래스 내부 구성요소 및 클래스 간의 관계를 도식화하여 시스템의 특정 모듈이나 일부 또는 전체 구조를 나타낸다.



출처: 집필진 제작(2021)
[그림 2-1] 유스케이스 다이어그램 예시



출처: 집필진 제작(2021)
[그림 2-2] 클래스다이어그램 예시

(1) 속성 (Attribute)

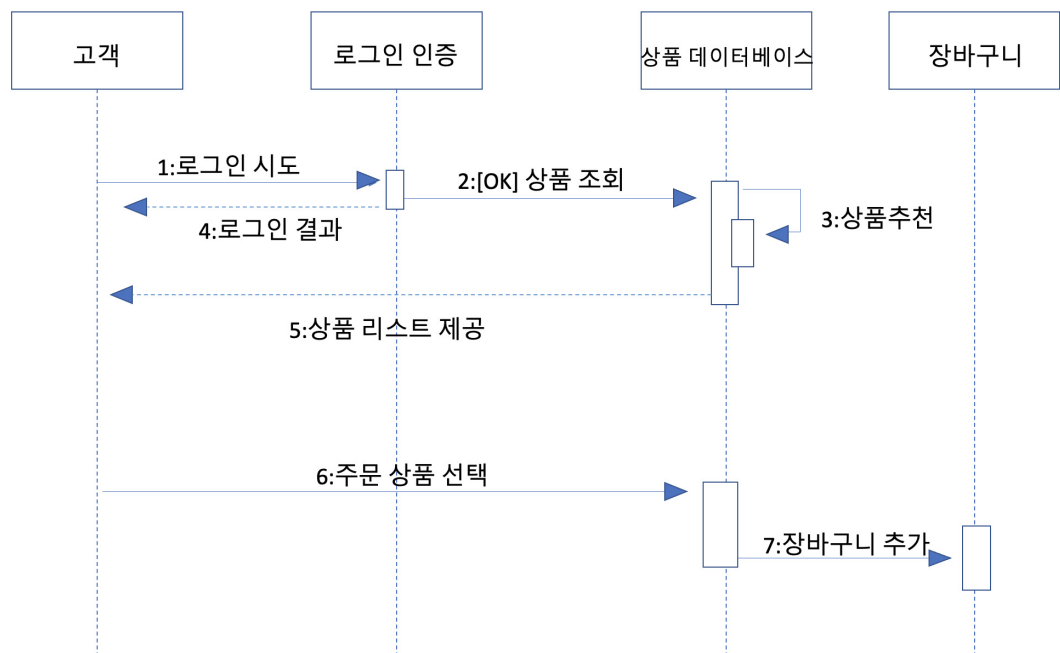
보통 명사형으로 많이 표현되며 실제 구현된 클래스를 기준으로 어트리뷰트 (Attribute)는 클래스의 멤버 변수로 사용된다.

(2) 행동 (Operation)

다이어그램에서 가장 하단에 위치하고 있으며 해당 클래스가 수행할 수 있는 오퍼레이션(Operation)들을 가진다.

3. 시퀀스 다이어그램

문제 해결을 위한 객체를 정의하고 객체 간의 상호작용 메시지 시퀀스를 시간의 흐름에 따라 나타내는 다이어그램이다.



출처: 집필진 제작(2021)

[그림 2-3] 시퀀스다이어그램 예시

(1) 활성 객체

시스템의 행위자 혹은 시스템 내의 유효한 객체 라이프라인(Life Line)을 가진다.

(2) 메시지 (Message)

서로 다른 객체 간의 상호작용 혹은 의사소통 통신을 정의하는 요소이다. 자신의 객체 라이프라인으로부터 다른 객체 라이프라인까지 선 + 화살표로 표시되며 메시지는 그 선의 위에 표시한다.

(3) 활성 박스 (Activation Box)

객체 라이프라인 위에 그려지는 박스로 이 박스 위에서 객체의 호출이 이루어진다. 객체의 특정 메소드 실행 혹은 정보 처리가 실행되고 있거나 다른 객체의 메소드가 종료되기를 기다린다는 것을 나타낸다.

재료·자료

- 개발 요구사항 정의서
- 구조적 프로그래밍 설계서
- 기능 명세 설계서
- 업무 정의서(업무 프로세스, 업무 처리 기준 등)

기기(장비 · 공구)

- 전산 장비(컴퓨터, 프린터, 빔 프로젝터, 네트워크 장비 등)
- OA 도구: 워드 프로세서, 스프레드시트, 프레젠테이션 도구 등
- CASE 도구: HIPO, SREM, PSL/PSA 등

안전 · 유의 사항

- 설계를 지원하는 도구들을 다룰 수 있어야 한다.
- 객체지향 설계의 원리에 대하여 이해할 수 있어야 한다.

수행 순서

① 소프트웨어 개발 요구사항을 확인한다.

기능 모델링을 수행하기 위해서는 사용자 혹은 시스템의 요구사항을 확인하여 필요 기능들을 추출해야 한다.

1. 시스템 요구사항 기능을 확인한다.

사용자 혹은 시스템이 요구하는 요구사항의 목록을 확인한다. 요구사항 목록은 크게 기능 요구사항과 비기능 요구사항으로 이루어져 있다. 일반적으로 시스템 요구사항에는 기능, 보안, 성능 데이터, 테스트, 품질, 제약 사항, 프로젝트 관리, 프로젝트 지원 등 다양한 시스템 요구사항이 분류되어 있다. 아래 사례는 기능 모델 다이어그램 작성을 위해 요약한 요구사항 목록의 예이다.

〈표 2-1〉 근태 관리 시스템의 기능 요구사항 분류 사례

요구사항 분류	요구사항 고유 번호	요구사항 명칭
기능 - SFR (System Function Requirement)	SFR-001	출퇴근 현황 조회
	SFR-002	관리자 기능
	SFR-003	외출 신청 기능
	SFR-004	유연 근무제 관리 기능
	SFR-005	특근 신청 관리 기능
	SFR-006	최종 점검 관리 기능
	SFR-007	초과 근무 관리 기능
	SFR-008	권한 관리 기능

출처: 교육부(2017). 기능모델링(LM2001020220_19v2). 한국직업능력개발원. p.22.

2. 시스템 상황을 분석한다.

문제 기술서(Problem Statement)에 시스템 요구사항을 명확히 기술해야 한다. 다음은 근태 관리 시스템의 문제 기술서 사례이다.

(1) 출/퇴근 관리 기능

(가) 로그인 후 출/퇴근 관리 화면의 출근 버튼을 통해 출근 시간을 입력한다.

(나) 퇴근 시 출/퇴근 관리 화면의 퇴근 버튼을 통해 퇴근 시간을 입력한다.

(다) 출/퇴근 관리 화면을 통해 기간별 출/퇴근 현황을 확인한다.

(2) 근태 현황 관리 기능

(가) 관리자가 그룹별로 지사, 본사, 전체, 개인의 근태 현황을 조회한다.

(나) 관리자가 부서별, 개인별, 일자별로 근태 현황 검색이 가능하다.

(3) 초과 근무 관리 기능

(가) 로그인 후 초과 근무 관리 화면을 통해 초과 근무를 신청한다.

(나) 초과 근무는 관리자의 검증이 필요하다.

(4) 특근 신청 관리 기능

(가) 로그인 후 특근 신청 관리 화면을 통해 특근을 신청한다.

(나) 특근 신청은 결재를 통해 승인자에게 승인을 받아야 한다.

(5) 승인 및 반려 기능

(가) 로그인 후 결재 관리 화면을 통해 결재 현황을 확인한다.

(나) 요청된 결재 정보를 승인 및 반려할 수 있다.

② 설계된 다이어그램을 분석한다.

1. 유스케이스 다이어그램을 분석한다.

(1) 액터(Actor)를 식별한다.

액터는 시스템과 작용하는 외부 객체이다. 액터는 사람이 될 수도 있고 외부 시스템이 될 수도 있다. 다음은 근태 관리 시스템 사례에서 식별된 액터들이다. 이때 승인자는 직원일 수도 있다.



출처: 집필진 제작(2021)

[그림 2-4] 근태 관리 시스템의 액터 사례

(가) 프라이머리 액터(Primary Actor)를 식별한다.

시스템을 사용함으로써 이득을 얻는 액터로 보통 외부 객체가 사람일 때를 말하며 사람 모양을 이용해 표기한 것을 확인한다.

(나) 세컨더리 액터(Secondary Actor)를 식별한다.

프라이머리 액터가 이득을 얻기 위해 도움을 주는 액터로 보통 외부 시스템을 의미하며 박스에 <<actor>>를 입력하여 표기한 것을 확인한다.

2. 관계(Relationship)를 식별한다.

(1) 포함 관계(필수적 관계)를 식별한다.

여러 유스케이스에서 중복되는 경우 분리하여 새로운 유스케이스를 만드는 경우인 포함 관계를 확인한다. 이런 부분은 상속 관계로 구현할 수 있다. 원래의 유스케이스에서 새롭게 만들어진 유스케이스 방향으로 화살표를 점선으로 연결하고 <<include>>를 확인한다.

(2) 일반 관계를 식별한다.

구체적인 유스케이스에서 일반화된 유스케이스 방향으로 끝부분이 삼각형의 테두리로 표현된 화살표를 실선으로 연결하여 표현한 것을 확인한다.

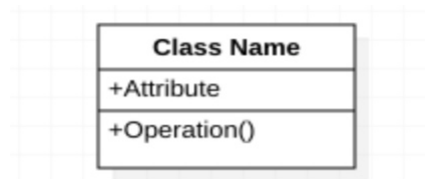
(3) 확장 관계를 식별한다.

유스케이스가 특정한 조건을 만족하는 경우에만 실행되는 단계를 확장 관계로 표

현하며, 표기 방법은 특정한 조건이 만족하는 경우에만 실행되는 단계를 따로 유스케이스로 만들고, 원래의 유스케이스 방향으로 화살표를 점선으로 연결한 후 <<extend>>를 확인한다.

3. 클래스 다이어그램을 분석한다.

클래스 다이어그램은 클래스 네임(Class name)과 어트리뷰트 그리고 오퍼레이션으로 구성되어 있다. 기본적으로 이 세 가지가 다이어그램의 구성 요소로 존재하고 클래스 네임, 어트리뷰트, 오퍼레이션 순으로 위에서 아래로 배치된다.



출처: 집필진 제작(2021)
[그림 2-5] 클래스 다이어그램 예시

(1) 속성을 확인한다.

클래스 네임과 오퍼레이션 사이의 중간층에 위치하며 명사형으로 많이 표현된다. 실제 구현된 클래스를 기준으로 어트리뷰트는 클래스의 멤버 변수로 구현한다.

(2) 행동을 확인한다.

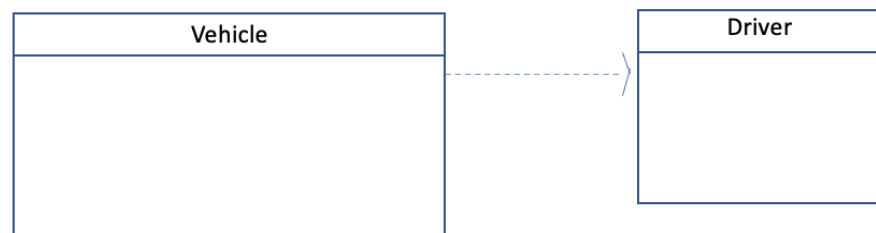
다이어그램의 가장 하단에 위치하고 있으며 해당 클래스가 수행할 수 있는 오퍼레이션들을 가진다. 클래스 메소드(Class Method)로 구현할 수 있는 것을 확인한다.

(3) 연관을 확인한다.

클래스 간의 논리적/물리적 관계를 나타낸다. 선과 선 끝의 머리 모양을 통해 관계를 구분한다.

(가) Dependency(연관)를 확인한다.

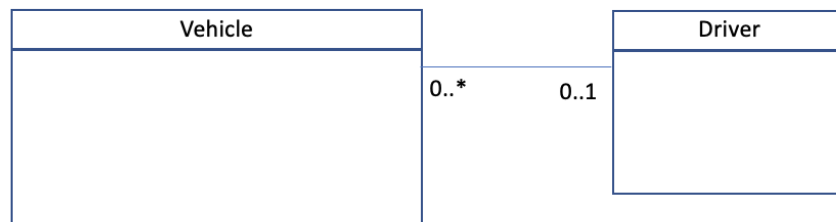
클래스가 다른 클래스와의 의존성을 가지고 있는 경우 의존 클래스의 상태 변화는, 즉 이를 참조하고 있는 클래스의 상태 변화(의존)를 의미한다.



출처: 집필진 제작(2021)
[그림 2-6] Dependency 예시

(나) Association / Directed Association(연관)을 확인한다.

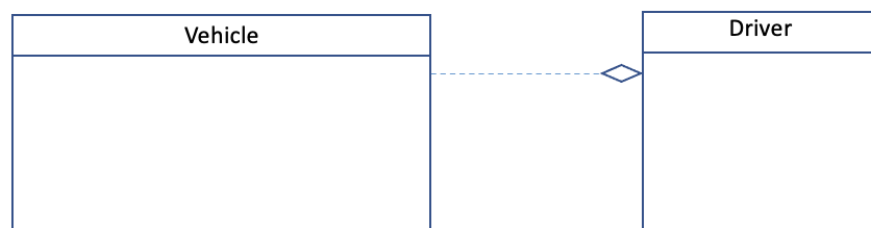
Association은 객체 간의 연관 관계를 나타낸다. 객체 지향에서 Association은 보통 인스턴스 변수로의 참조를 의미한다. 즉, 상대 클래스의 인스턴스를 어트리뷰트로 가질 때 Association Relationship(연관 관계)이 있다고 한다. 이때의 관계는 명료한 모델링을 필요로 하며, 보통 양방향성을 나타내는 Association(Standard)과 단방향성을 나타내는 Directed Association이 사용 되는 것을 확인한다.



출처: 집필진 제작(2021)
[그림 2-7] Association 예시

(다) Aggregation(집합 연관)을 확인한다.

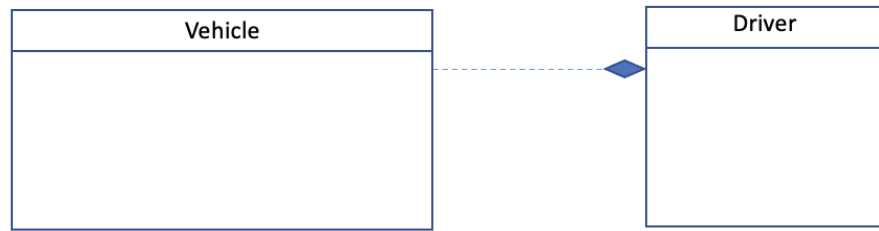
Association 개념에서 조금 더 확장된 개념으로 전체와 부분의 관계를 나타낸다. 두 개의 클래스가 서로 연관 관계를 가지며 전체 - 부분 관계를 가질 때 Aggregation으로 표기한 것을 확인한다.



출처: 집필진 제작(2021)
[그림 2-8] Aggregation 예시

(라) Composition(합성 연관)을 확인한다.

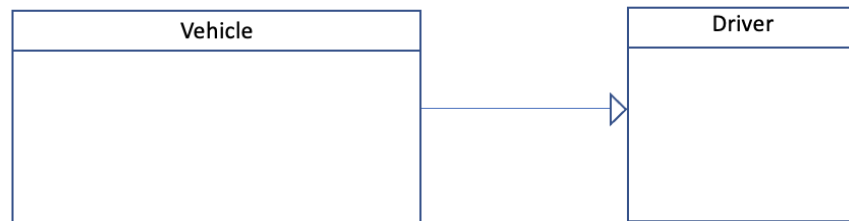
Composition은 Aggregation처럼 전체와 부분의 관계를 띄고 있지만 부분에 해당하는 클래스의 라이프사이클이 전체를 의미하는 클래스의 라이프사이클과 동일하다는 점에서 차이점을 보이는 것을 확인한다.



출처: 집필진 제작(2021)
[그림 2-9] Composition 예시

(마) Generalization(일반화)을 확인한다.

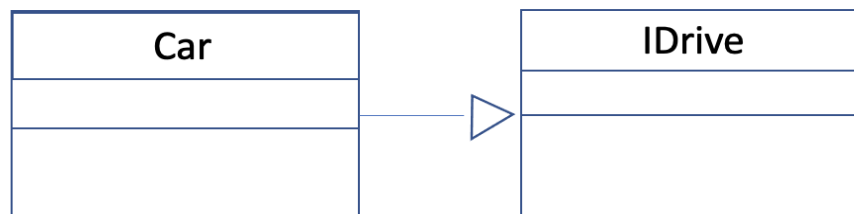
Generalization은 "is A" 관계를 가지는 두 클래스 간의 관계를 의미한다. 객체 지향에서는 Generalization(일반화)를 부모 클래스와 자식 클래스간의 "상속" 관계로 해석된다. 자식 클래스들은 부모 클래스가 가지는 공통 특징(속성 or 행동)을 가진 것을 확인한다.



출처: 집필진 제작(2021)
[그림 2-10] Generalization 예시

(바) Realization(실체화)을 확인한다.

객체지향에서 Realization은 명세화된 혹은 정의만 된 오퍼레이션을 실체화하는 것을 의미한다. 즉, 인터페이스(Interface)로 명세화된 기능을 실제 기능으로 구현하는 것을 말하는데, 인터페이스의 Implement로 확인하면 된다.

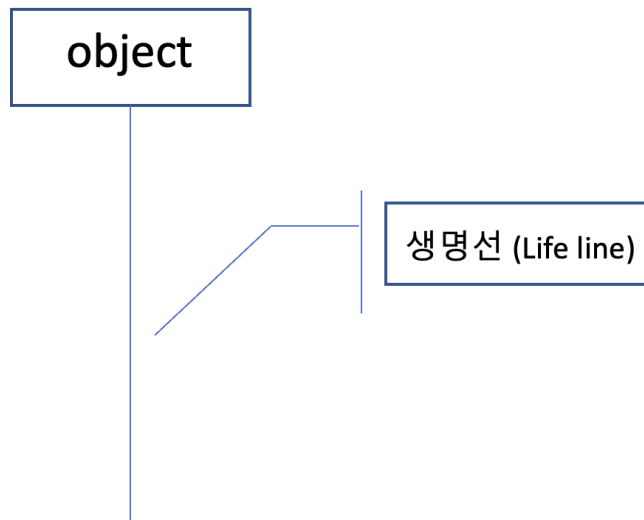


출처: 집필진 제작(2021)
[그림 2-11] Realization 예시

4. 시퀀스 다이어그램을 분석한다.

(1) 활성 객체를 확인한다.

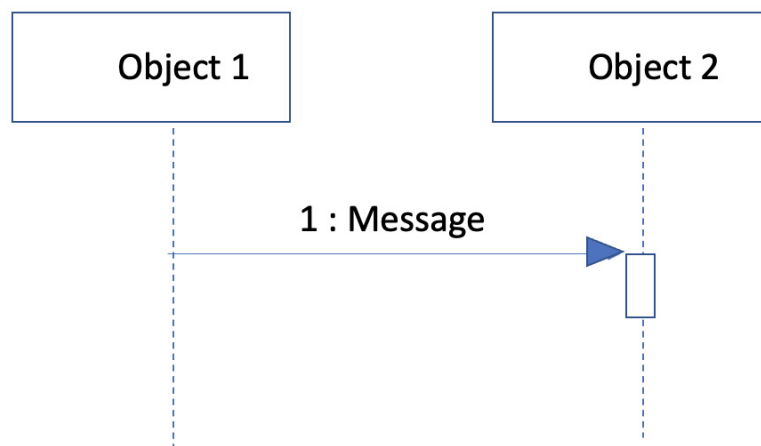
시스템의 행위자 혹은 시스템 내의 유효한 객체인 라이프라인을 가진 것을 확인한다.



출처: 집필진 제작(2021)
[그림 2-12] Life line 예시

(2) 메시지(Message)를 확인한다.

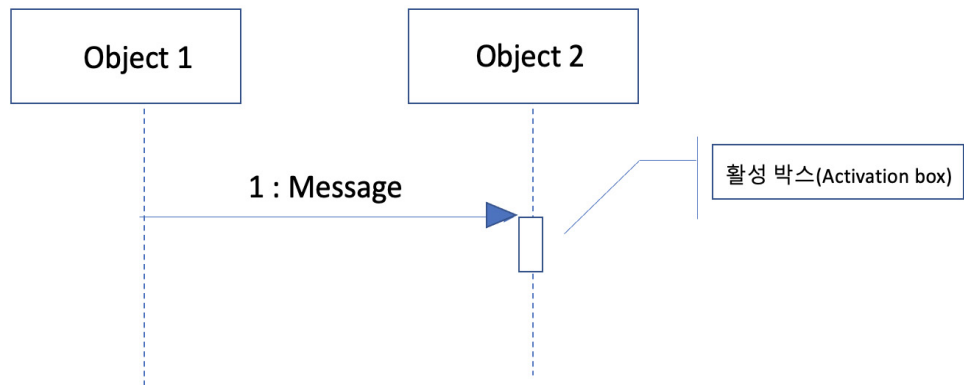
서로 다른 객체 간의 상호작용 혹은 의사소통 통신을 정의하는 요소로, 하나의 객체 라이프라인으로부터 다른 객체 라이프라인까지 선 + 화살표로 표시되며 메시지는 그 선의 위에 표시한 것을 확인한다.



출처: 집필진 제작(2021)
[그림 2-13] message 예시

(3) 활성 박스(Activation Box)를 확인한다.

객체 라이프라인 위에 그려지는 박스로 이 박스 위에서 객체의 호출이 이루어진다. 객체의 특정 메소드 실행 혹은 정보 처리가 실행되고 있거나 다른 객체의 메소드가 종료되기를 기다린다는 것을 나타낸 것을 확인한다.



출처: 집필진 제작(2021)
[그림 2-14] Activation Box 예시

2-2. 객체지향 프로그래밍 언어 활용

학습 목표

- 객체지향 언어를 활용하여 애플리케이션을 작성할 수 있다.
- 작성된 애플리케이션의 오류를 식별하고 수정할 수 있다.

필요 지식 /

① 객체지향 프로그래밍 언어의 개요

프로그래밍에서 필요한 데이터를 추상화시켜서 상태와 행위를 가진 객체를 만들고, 그 객체들 간의 유기적인 메시지를 통해 상호작용 로직을 구성하는 프로그래밍 방법이다.

1. 객체지향 프로그래밍 언어 구성요소

객체 프로그래밍은 기본적으로 클래스 구조에서 시작한다. 클래스는 객체지향 개념에서 객체를 정의하는 틀로서 많은 객체지향 프로그래밍 언어의 기본 구조이다. 클래스의 기본 구성요소는 변수와 메소드이다.

(1) 클래스(Class)

같은 종류의 집단에 속하는 속성과 행위를 정의한 것으로 객체지향 프로그램의 기본적인 사용자 정의 데이터형이라고 할 수 있다.

(가) 일반적으로 클래스 이름과 소스 파일명은 동일하다.

(나) 프로그램은 여러 클래스로 구성되며 실행을 위해서는 `main()` 메소드를 사용한다.

(2) 객체(Object)

클래스의 인스턴스(Instance)로 자신 고유의 데이터를 가지며 클래스에서 정의한 행위를 수행할 수 있다.

(가) `main()` 메소드는 프로그램이 실행하는 진입점이고 실제 클래스를 사용하려면 `new()` 연산을 통해 인스턴스를 생성한다.

(나) `main()` 에서 클래스부에 선언된 변수를 접근할 수 없으며 인스턴스를 통해 사용한다.

(다) 인스턴스에서 변수와 메소드 사용은 인스턴스명 다음에 변수명, 인스턴스명, 메소드명과 같은 형식으로 사용한다.

(3) 메소드(Method)

클래스로부터 생성된 객체를 사용하는 방법으로 특정 기능을 수행하면서, 객체에 명령을 내리는 기능을 가지고 있다.

(4) 속성(Property)

한 클래스 내에 속한 객체들이 가지고 있는 데이터 값을 단위별로 정의한 것으로 성질이나 분류 또는 현재 상태 등에 대한 표현값을 가지고 있다.

클래스, 메소드 선언 예시

```
//클래스 선언
public class MyClass {
    //변수 선언
    int num1;
    Message msg;

    //메소드
    public void printName(String name) {
        ...
    }

    //메소드
    public Message getMessage() {
        ...
    }

    //메인 메소드
    public static void main(String[] args) {
        //클래스 인스턴스 생성
        MyClass mc = new MyClass();
        ..
    }
}
```

2. 접근 지정자

(1) public

모든 접근을 허용하는 접근자로 공개적으로 오픈할 때 사용하는 접근자이다

(2) protected

자기 자신 클래스 및 상속받은 자식 클래스에서의 접근을 허용하는 접근자이다.

(3) private

자기 자신 클래스 내부의 메소드만 접근을 허용하는 접근자이다.

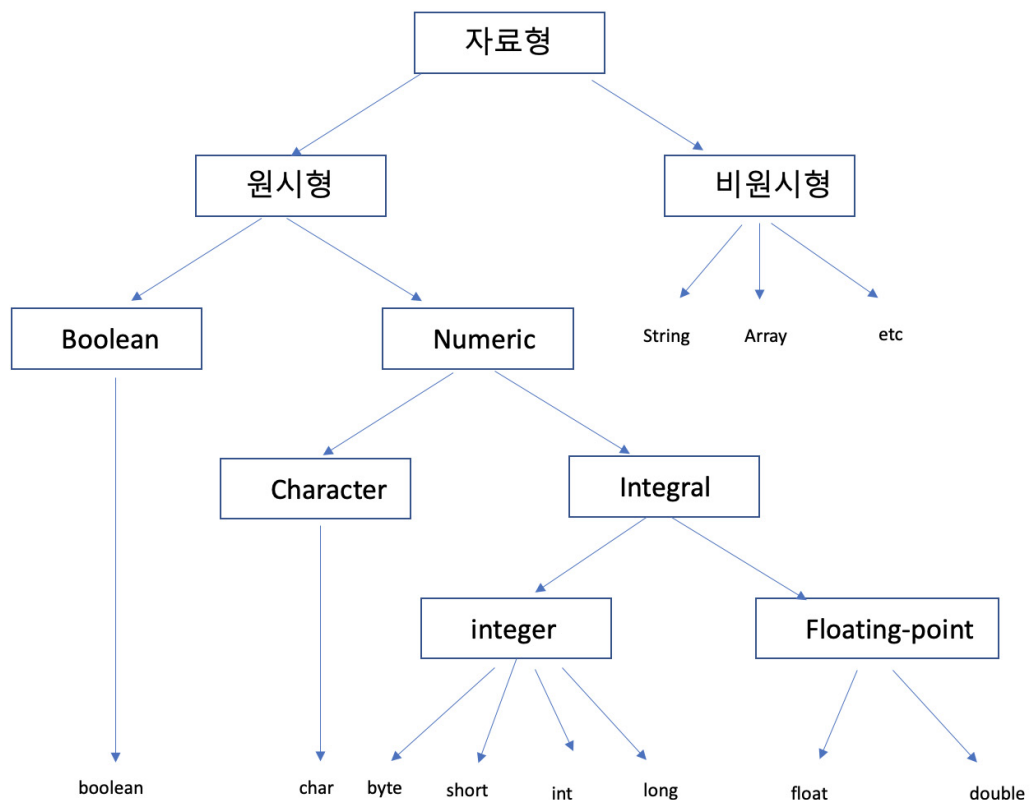
(4) default

접근 지정자를 명시하지 않는 경우이다. 같은 패키지에서는 모두 접근 가능하지만 다른 패키지일 때는 접근이 불가능하다.

② 객체지향 프로그래밍 언어 기본 문법

1. 데이터 타입

원시형 자료는 자바(Java)에서 기본적으로 사용하고 있는 char, boolean, byte, short, int, long가 포함되고, 비원시 자료형의 경우는 원시형 자료형을 묶어서 만든 것으로 단독의 자료형을 사용한다.



출처: 집필진 제작(2021)
[그림 2-15] 객체지향 데이터타입

2. 변수와 메소드

변수는 데이터를 저장하기 위한 메모리 공간에 대한 이름으로, 저장할 데이터의 크기를 알아야 필요한 공간을 확보할 수 있다. 객체지향 프로그램 언어에서는 클래스

타입을 자료형으로 사용할 수 있으며, 최근 사용하는 언어들의 경우 메모리 공간의 크기를 계산하기 위한 용도보다는 타입을 구분하는 용도로 사용하고 있다.

(1) 변수 선언 방법

변수 선언 방법

```
int num1 = 10;  
String msg = "Hello";  
Member member = new Member();
```

접근제어자 선언 방법

[접근제어자] 타입 변수명

(가) 접근제어자: 변수의 접근 범위를 지정한 것으로 public, private, protected, default 등을 이용할 수 있다.

(나) 타입: 자료형으로 명시적으로 타입을 지정한 것으로 사용할 수 있는 데이터형을 선언하는 곳이다.

(2) 변수 유형

(가) 멤버 변수(Member Variable)

클래스부에 선언된 변수들로 객체의 속성에 해당하고, 인스턴스 변수와 클래스 변수로 구분된다.

(나) 인스턴스 변수(Instance Variable)

클래스가 인스턴스될 때 초기화되는 변수로서 인스턴스를 통해서만 접근할 수 있다.

(다) 매개 변수(Parameter)

매개 변수는 메소드에 인자로 전달되는 값을 받기 위한 변수로서 메소드 내에서 지역변수처럼 사용된다.

(라) 지역변수(Local Variable)

메소드 내에서 선언된 변수로서 멤버 변수와 동일한 이름을 가질 수 있으며 지역적으로 우선일 때 사용하게 된다.

(마) 클래스 변수(Class Variable)

static으로 선언된 변수로, 인스턴스 생성 없이 클래스 이름의 변수명으로 사용 가능하고, main() 메소드에서 참조 가능하다.

변수 유형

```
public class variables {  
    //멤버 변수, 인스턴스 변수  
    int num1;  
  
    //멤버 변수, 클래스 변수  
    static int num2;  
  
    //매개 변수  
    public void printName(String name) {  
        //지역변수  
        String prtMsg = name + " Hello";  
        System.out.println(prtMsg);  
    }  
  
    public static void main(String[] args) {  
        //인스턴스 생성  
        variables mc = new variables();  
        //인스턴스 변수 사용  
        mc.num1 = 100;  
        //클래스 변수 사용  
        variables.num2 = 50;  
  
        mc.printName("홍길동");  
  
        System.out.printf("%d, %d,mc.num1, variables.num2);  
    }  
}
```

(3) 메소드

메소드는 특정 객체의 동작이나 행위를 정의한 것으로 클래스의 주요 구성요소이다.

메소드 형식

```
[접근제어자] 리턴 타입 메소드명([인자..]) {  
  
}
```

(가) 접근제어자: 메소드의 접근 범위를 지정한다.

(나) 리턴 타입: 함수 내에서 결과를 보내는 위치이다. 리턴이 없는 경우에는 void를 사용한다.

(4) 연산자 (Operator)

연산자는 프로그램에서 변수의 값들을 계산하기 위해 사용한다. 대입연산, 사칙연산, 비트연산, 관계연산 등이 있다.

③ 디자인 패턴

디자인 패턴은 객체지향 프로그래밍 시 발생하는 여러 가지 문제에 대한 설계 사례를 분석하여 공통적으로 발생하는 문제를 해결하기 위한 방법론으로, 여러 설계들로 분류하고, 각 문제 유형별로 가장 적합한 설계를 일반화한 패턴이다.

1. 디자인 패턴 구조

(1) 콘텍스트(Context)

문제가 발생하는 여러 상황을 기술한다. 패턴이 적용될 수 있는 상황을 나타내고, 패턴이 유용하지 못한 상황을 나타내기도 한다.

(2) 문제(Problem)

패턴이 적용되어 해결될 필요가 있는 여러 디자인 이슈들을 기술한다. 이때에는 여러 제약 사항과 영향력도 문제 해결을 위해 고려해야 한다.

(3) 해결(Solution)

문제를 해결하도록 설계를 구성하는 요소들과 그 요소들 사이의 책임 관계, 협력 관계를 기술한다. 해결은 다양한 상황에 적용할 수 있는 일종의 템플릿이다.

2. 디자인 패턴의 종류

(1) GoF 디자인 패턴

GoF 디자인 패턴은 23가지의 디자인 패턴이 정리되어 있고, 구현 방식에 따라 생성(Creational), 구조(Structural), 행위(Behavioral) 3가지로 분류한다.

(2) GoF 디자인 패턴의 분류

생성 패턴에는 Abstract Factory, Builder, Factory Methods, Prototype, Singleton이 있으며, 구조 패턴에는 Adaptor, Bridge, Composite, Decorator, Facade, Flyweight, Proxy이 있다. 행위 패턴에는 Chain of Responsibility, Command, Interpreter, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor가 있다.

〈표 2-2〉 디자인 패턴의 유형

생성 패턴	구조 패턴	행위 패턴
		Chain of Responsibility
		Command
	Adaptor	Interpreter
Abstract Factory	Bridge	Interpreter
Builder	Composite	Mediator
Factory Methods	Decorator	Memento
Prototype	Facade	Observer
Singleton	Flyweight	State
	Proxy	Strategy
		Template Method
		Visitor

수행 내용 / 객체지향 프로그래밍 언어 활용하기

재료 · 자료

- 프로그램 설계서, 유스케이스 다이어그램, 클래스 다이어그램, 시퀀스 다이어그램
- 요구사항 정의서
- 시스템 아키텍처

기기(장비 · 공구)

- 컴퓨터, 네트워크 장비, 프린터, 빔 프로젝트
- CASE 도구(SW 개발 지원 도구, SW 테스트 지원 도구 등)
- 형상관리 도구

안전 · 유의 사항

- 공통 모듈 설계 산출물에 대한 이해가 선행되어야 한다.
- 프로그램 개발 경험과 오픈소스 경험이 필요하다.

수행 순서

① 후보 객체를 도출한다.

적합성 여부를 떠나 문제 기술서나 배경 지식, 상식 등을 동원하여 가능한 모든 후보 클래스들을 파악한다. 적합성을 판단하여 부적절한 클래스는 제거하고, 클래스 대신 속성이나 연산으로 대체 가능한 경우 대체하여 도출한다.

1. 명사 분석 방법을 이용한다.

설계서를 바탕으로 명사를 찾는다. 이미 명명이 완료된 대상은 어느 정도 개념이 명확하기 때문에 이름이 붙은 것이므로 잠정적으로 클래스로 간주될 수 있다.

2. 문제 영역에 대한 배경 지식을 활용한다.

설계서에 나오지 않는 것이라도 관련 지식과 상식을 바탕으로 추가적으로 클래스를 도출한다. 개발자는 시스템의 요구사항을 파악하는 데 도움을 줄 수 있는 모든 유형의 정보를 이용한다.

3. 전형적인 유형의 클래스를 도출한다.

시스템 유형에 관계없이 일반적으로 존재할 수 있는 전형적인 클래스를 찾는다.

4. 유스케이스 기반으로 클래스를 도출한다.

유스케이스를 분석하여 여러 유형의 클래스 도출이 가능하다.

(1) 경계 클래스(Boundary Class)를 도출한다.

개발할 시스템과 시스템 외부와의 인터페이스 역할을 하는 클래스

(2) 실체 클래스(Entity Class)를 도출한다.

영속적인 정보와 그 정보에 대한 조작 기능을 제공하는 클래스

(3) 제어 클래스(Control Class)를 도출한다.

제어 로직 및 비즈니스 로직을 제공하는 클래스

5. 디자인 패턴을 고려하여 클래스를 설계한다.

(1) 생성(Creational) 패턴을 고려한다.

객체 생성에 관련된 패턴으로 객체의 생성과 조합을 캡슐화해 특정 객체가 생성되거나 변경되어도 프로그램 구조에 영향을 크게 받지 않도록 유연성을 제공하는 패턴을 적용해본다.

(가) 추상 팩토리(Abstract Factory)

구제적인 클래스에 의존하지 않고 서로 연관되거나 의존적인 객체의 조합들을 만드는 인터페이스를 제공하는 패턴이다.

(나) 팩토리 메소드(Factory Method)

객체 생성 처리를 서브클래스로 분리하여 처리하도록 캡슐화하는 패턴이다.

(다) 싱글턴(Singleton)

전역변수를 사용하지 않고 객체를 하나만 생성하여, 생성된 객체를 한 번만 어디에서든지 참조할 수 있도록 하는 패턴이다.

(2) 구조(Structural) 패턴을 고려한다.

클래스나 객체를 조합해 더 큰 구조를 만드는 패턴이다. 서로 다른 인터페이스를 지닌 객체를 묶어 단일 인터페이스를 제공하거나 객체들을 서로 조합하여 새로운 기능을 제공하는 패턴이다.

(가) 콤포짓(Composite)을 고려한다.

여러 개의 객체로 구성된 복합 객체와 단일 객체를 클라이언트에서 구별 없이 사용할 수 있게 해주는 패턴

(나) 데코레이터(Decorator)을 고려한다.

객체의 결합을 통해 기능을 동적으로 구현하여 유연하게 확장할 수 있게 해주는 패턴

(다) 행위(Behavioral) 패턴을 고려한다.

객체나 클래스 사이의 책임 분배나 알고리즘에 관련된 패턴이다. 한 객체가 혼자 수행할 수 없는 작업을 여러 개의 객체로 어떻게 분배하는지와 객체 사이의 결합도를 최소화하는 것에 중점을 두는 패턴이다

1) 옵서버(Observer)

한 객체의 상태 변화를 감지하여 다른 객체의 상태도 연동되도록 일대다 객체에 의존 관계를 구성하는 패턴

2) 스테이트(State)

객체의 상태에 따라 객체의 행위 내용을 변경해줄 수 있도록 구현되는 패턴

3) 스트레티지(Strategy)

행위를 클래스로 캡슐화해 동적으로 행위를 자유롭게 바꿀 수 있게 구현해주는 패턴

4) 템플릿 메소드(Template Method)

어떤 작업을 처리하는 일부분을 서브클래스로 캡슐화하여 전체 작업을 수행하는 패턴으로, 구조는 바꾸지 않으면서 특정 단계에서 수행하는 내역을 바꾸는 패턴

5) 커맨드(Command)

실행될 기능을 캡슐화함으로써 제시된 여러 기능을 실행할 수 있도록 하여 재사용성을 높인 클래스로 설계하는 패턴

② 객체지향 프로그래밍 기법으로 구현한다.

1. 객체프로그래밍 기본 구조를 잡는다.

main 메소드에서 기본적으로 실행을 한다.

JAVA 기본 형식

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello World!!");  
    }  
}
```

2. 클래스를 생성한다.

클래스는 new 키워드를 이용하여 객체를 생성할 수 있고, 생성된 객체는 메모리에 적재되어 사용할 수 있다.

3. 상속을 구현한다.

상속은 바로 클래스 간의 상하 관계로 추상적인 슈퍼클래스(Super Class) 혹은 부모

클래스로부터 서브클래스(Sub Class) 혹은 자식 클래스를 만드는 것으로 상속이라는 관계를 통해 계층 구조를 형성하게 한다.

(1) 클래스 상속 방법을 고려한다.

(가) 슈퍼클래스에서 정의된 필드와 메소드를 상속받는다.

(나) 새로운 필드나 메소드를 추가할 수 있다.

(다) 슈퍼클래스에서 이어받은 메소드를 수정할 수 있다.

(라) 같은 슈퍼클래스를 상속하는 모든 서브클래스는 타입 호환이 된다.

(2) 상속 구현을 한다.

(가) 구현 코드

상속 구현 예시

```
Class SubClass extends SuperClass {  
  
}
```

(나) 오버라이딩(Overriding)

슈퍼클래스로부터 상속받은 메소드를 다시 정의하는 것을 말한다. 메소드의 이름과 리턴 타입, 인자 등이 모두 동일해야 하며, 다를 경우 새로운 메소드가 추가되는 형식이 된다.

오버라이딩 구현 예시

```
class Animal{  
    void bark(){  
        System.out.println("bow");  
    }  
}  
  
class Dog extends Animal{  
    void bark(){  
        System.out.println("woof");  
    }  
}  
  
class Cat extends Animal {  
    void bark(){  
        System.out.println("ultra");  
    }  
}
```

4. 추상화 구현을 한다.

(1) 추상 클래스(Abstract Class)을 구현한다.

추상 클래스는 추상 메소드(Abstract Method)를 포함하고 있는 클래스이다. 추상 메소드란 구체적이지 않은 내용으로 정의한 메소드로 추상 메소드로 정의된 메소드는 서브 클래스에서 반드시 오버라이딩을 통해 구현해야 한다.

추상 클래스 구현 예시

```
abstract class Pet {  
    abstract void bark() {} ;  
}
```

(2) 인터페이스(Interface)를 구현한다.

추상 클래스와 유사하게 구체적이지 않은 내용을 정의한 것으로 상수와 추상 메소드로만 선언된 형태를 말한다. 구현 내용이 없는 메소드로만 구성이 되기 때문에 자체로는 아무런 기능을 하지 않지만, 이후 구현될 클래스들을 연결하여 사용할 수 있는 기반 구조를 제공한다.

(3) 패키지과 제어자를 고려한다.

(가) 패키지

패키지는 자바 클래스들을 구분하기 위해 사용한다. package와 같은 키워드를 이용해 패키지를 선언한다.

(나) 제어자

제어자는 변수, 클래스, 메소드의 선언부에 사용되어 부가적인 의미를 부여한다. public, static 같은 키워드들이 여기에 해당된다. 이러한 제어자에는 클래스의 접근 범위와 관련된 접근 제어자(Access Modifier)와 일반 제어자가 있다.

③ 객체지향 프로그래밍 오류 점검 및 디버깅을 한다.

프로그램에서 문제가 발생하는 것을 에러(error)라고 하며, 보통 에러는 컴파일 과정에 발생하는 컴파일 에러와 실행 중 발생하는 런타임 에러로 구분한다.

1. 객체지향 프로그래밍을 테스트한다.

(1) JUnit을 이용하여 테스트한다.

JUnit은 자바 프로그래밍 언어용 단위 테스트 도구로 보이지 않고 숨겨진 단위 테스트를 끌어내어 정형화시켜 단위 테스트를 쉽게 해주는 테스트용 Framework이다.

(가) @Test 메소드가 호출할 때마다 새로운 인스턴스가 생성되어 독립적인 테스트가 가능하다.

(나) assert 문으로 테스트 케이스의 수행 결과를 판별할 수 있다.

(다) 결과는 성공(녹색), 실패(붉은색) 중 하나로 표시된다.

(라) 테스트 결과를 확인하는 것 이외 최적화된 코드를 유추해내는 기능도 제공한다.

2. 객체지향 예외 처리를 한다.

프로그램 실행 중 발생할 수 있는 예외적인 상황을 대비한 코드를 미리 만들어두는 것으로 문제 발생 시 좀 더 안전한 구조를 제공하며 문제 해결에 필요한 정보 등을 제공할 수 있다.

(1) 예외 상황을 처리한다.

(가) 특정 파일을 로딩하려고 할 때 파일이 해당 경로에 없는 경우

(나) 네트워크로 서버에 접속하는 경우 서버가 다운되었거나 네트워크 연결에 문제가 있는 경우

(다) 문자열 데이터를 읽어 객체의 참조를 구하는 경우

(라) 정수를 0으로 나누는 경우

(2) 예외 처리 유형을 고려한다.

(가) Checked Exception - Exception 클래스를 상속받아 구현된 클래스로, 컴파일러에 의해 강제로 예외 처리가 요구됨

(나) Unchecked Exception - RuntimeException 클래스를 상속받아 구현된 클래스로, 강제로 예외 처리가 요구되지는 않지만 필요에 따라 예외 처리가 가능한 Exception.

3. 객체지향을 리팩토링한다.

(1) 리팩토링의 과정을 고민한다.

(가) 문제를 찾는다.

(나) 문제의 특징을 정한다.

(다) 솔루션을 디자인한다.

(라) 코드를 수정한다.

(2) 리팩토링 기법을 적용한다.

(가) Rename - 이름 재설정

변수, 메소드, 클래스들, 다른 자바 메소드의 이름이 혼동되지 않도록 수정한다.

(나) Move Class - 클래스의 이동

클래스를 적합한 패키지로 이동시킨다. 수정되는 구문들을 적합한 클래스로 이동하는 기법이다.

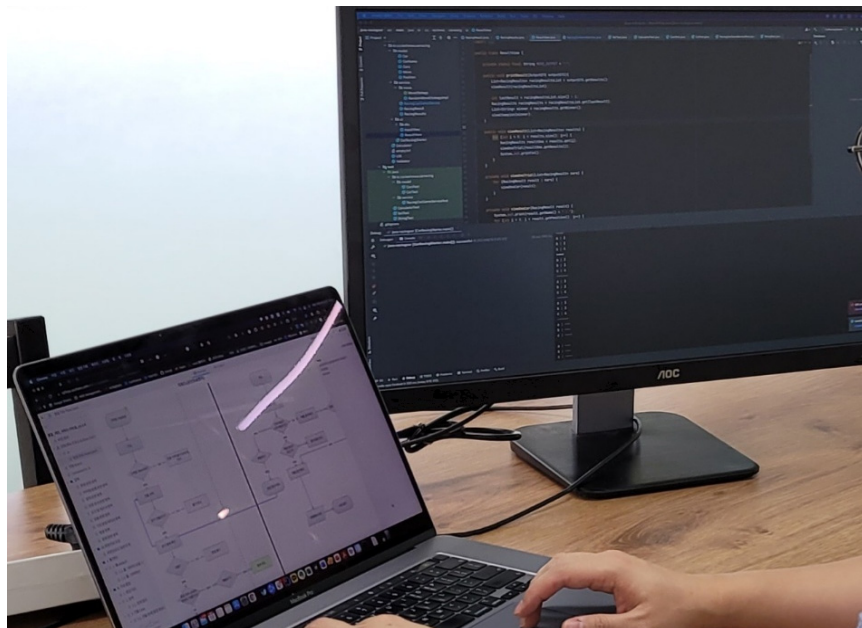
(다) Extract Method - 메소드 추출

긴 메소드를 쪼개서 가독성과 유지성을 높이기 위한 기법으로 한 가지 일을 수행하는 부분을, 서술적인 이름의 새로운 메소드로 만든다.

(라) Extract Superclass - 슈퍼클래스를 추출

구체화시키는 클래스들 중에서 공통된 메소드는 정의를 상위로 배치하고, 서브클래스에서 다양한 것들은 추상 메소드에 위치시킨다. 효율적인 코드로 재사용할 수 있도록 도와주고, 새로운 서브클래스들이 만들어지고 사용되도록 해준다.

(마) Replace Conditional with Polymorphism - 다형성을 조건으로 하는 것을 대체
if나 switch 같은 의사 결정 메소드를 만들어서 중복을 방지하기 위해 사용한다.



출처: 집필진 제작(2021)

[그림 2-16] 설계서를 바탕으로 객체지향 언어로 구현하는 모습

수행 tip

- 객체지향 설계 원칙을 이해하고 디자인 패턴도 학습을 하여 개발한다.
- 객체지향 개발 문법을 명확히 이해하고 있어야 한다.
- 구조화 프로그래밍과의 차이점을 이해하고 있어야 한다.

교수 방법

- 객체지향 설계 내용을 개발 언어와 도구를 활용하여 개발하는 절차에 대해 구체적으로 설명한다.
- 객체지향 프로그램 개발 시 개발 생산성을 높이기 위한 도구를 제시하여 효율적인 개발을 할 수 있도록 도구 활용 방안에 대해 설명한다.
- 객체지향 프로그램의 특성을 강조하여 다른 프로그램의 특징과 차이점에 대해 정리하여 설명한다.
- 개발된 객체지향 프로그램에서 제공하는 기능에 대한 리팩토링 작성 기법에 대해 정리하여 설명한다.
- 개발 모듈의 단위 테스트 수행을 위한 테스트 조건 명세화 절차와 내용에 대해 정리한 후 상세히 설명한다.

학습 방법

- 객체지향 프로그램 설계 내용을 개발 언어와 도구를 활용하여 개발하는 절차에 대해 숙지할 수 있도록 학습한다.
- 객체지향 프로그램 개발 시 개발 생산성을 높이기 위한 도구에 대하여 효율적인 개발을 할 수 있도록 실습한다.
- 객체지향 프로그램이 클라이언트 프로그램에 종속되지 않도록 유연성을 확보하는 방안에 대해 학습한다.
- 개발된 프로그램에서 발생된 오류를 리팩토링 기법을 이용하여 수정할 수 있도록 하며, 이를 실무에 활용할 수 있을 정도로 반복적으로 실습한다.
- 개발 모듈의 단위 테스트 수행을 위한 테스트 조건 명세화 절차와 내용에 대해 학습한다.

평가 준거

- 평가자는 학습자가 학습 목표를 성공적으로 달성하였는지를 평가해야 한다.
- 평가자는 다음 사항을 평가해야 한다.

학습 내용	학습 목표	성취수준		
		상	중	하
객체지향 프로그래밍 설계	- 응용소프트웨어 개발을 위하여 프로그램 설계서를 확인할 수 있다.			
객체지향 프로그래밍 언어 활용	- 객체지향 언어를 활용하여 애플리케이션을 작성할 수 있다.			
	- 작성된 애플리케이션의 오류를 식별하고 수정할 수 있다.			

평가 방법

- 포트폴리오

학습 내용	평가 항목	성취수준		
		상	중	하
객체지향 프로그래밍 설계	- 응용소프트웨어 개발을 위한 프로그램 설계 숙지 여부			
	- 객체지향 특징 파악 능력			
객체지향 프로그래밍 언어 활용	- 객체지향 프로그래밍 언어를 활용한 애플리케이션 작성 능력			
	- 작성된 애플리케이션의 오류 식별 및 수정 능력			

• 평가자 체크리스트

학습 내용	평가 항목	성취수준		
		상	중	하
객체지향 프로그래밍 설계	- 응용소프트웨어 개발을 위한 프로그램 설계서 파악 능력			
	- 객체지향 특징에 대한 숙지 여부			
객체지향 프로그래밍 언어 활용	- 객체지향 프로그래밍 언어를 활용한 애플리케이션 기능 숙지 여부			
	- 작성된 애플리케이션의 오류 식별 및 수정 능력			

피드백

1. 평가자 체크리스트

- 실습 과정을 체크리스트에 따라 평가한 후에 개선해야 할 사항 등을 정리하여 학습자에게 설명한다.
- 설계서를 분석하여 작성된 개발 모듈이 설계서대로 개발되었는지 확인하여 비정상 동작 시 그 문제점을 정리하여 개선 사항을 제시한다.
- 객체지향 특징에 대한 부족한 이해도를 보충할 수 있도록 실무 자료를 제시한다.
- 다양한 객체지향 설계도를 보여주면서 설계 이해 경험을 제시한다.

2. 포트폴리오

- 제출한 보고서를 평가한 후에 주요 사항에 대하여 의견을 제시한다.
- 개발 언어와 도구의 활용을 통하여 정상적으로 동작하는지를 보고 비정상 동작의 원인과 조치 방안에 대해 설명한다.
- 개발된 코드가 객체지향적으로 개발되었는지 평가하고 미비 사항을 설명한다.
- 개발 언어 문법의 오류를 확인하여 개선사항을 설명한다.
- 디버깅 방법에 대해 정확히 이해하고 있는지 확인하여 디버깅 방법을 명확하게 설명한다.

학습 1	구조적 프로그래밍 언어 활용하기
학습 2	객체지향 프로그래밍 언어 활용하기
학습 3	스크립트 언어 활용하기

3-1. 스크립트 언어 설계

학습 목표

- 응용소프트웨어 개발을 위하여 프로그램 설계서를 확인할 수 있다.

필요 지식 /

① 스크립트 언어 개요

스크립트 언어(Scripting Language)는 응용소프트웨어를 제어하는 컴퓨터 프로그래밍 언어이다. 초창기 스크립트 언어는 배치 언어(Batch Languages) 또는 작업 제어 언어(Job Control Language)라고도 불렸다.

1. 스크립트 언어 특징

스크립트 언어는 응용프로그램과 독립하여 사용되고 다른 응용프로그램의 언어와 다른 언어로 사용되어, 최종 사용자가 응용프로그램의 동작을 사용자의 요구에 맞게 수행할 수 있도록 해준다

(1) 인터프리터 언어

코드를 작성함과 동시에 인터프리터가 기계어로 번역하고 실행한다.

(2) 단순한 구문

스크립트 언어는 타 프로그래밍 언어에 비해 단순한 구문과 의미를 내포한다.

(3) 컴파일 시간 소요

스크립트 언어는 컴파일된 프로그램보다 실행 시간이 오래 걸린다. 스크립트 언어는 모든 명령어가 기본 명령어 처리기에 의해 직접 처리되지 못하고, 다른 프로그램에 의해 전 처리가 필요하기 때문이다.

(4) 신속한 활용

일반적으로 스크립트 언어는 빠르게 배우고 작성하기 위해 고안된 언어이다.

② 기능 설계서와 화면 설계서

1. 기능 설계서

기능을 작동하게 하기 위해서 정의된 기능 설계서에는 무엇을, 어떻게 만드는지 정의되어 있다. 설계서는 ‘무엇을’ 만들 것인지에 초점을 두어 작성이 되어 있고, 개발자는 기능 명세서를 보고 전체 구조와 설계를 함께 고려하여 ‘어떻게’를 개발한다.

〈표 3-1〉 기능 설계서 내용 예시

항목	설명
화면 입력 내용	입력 전, 입력 중, 입력 후 상태가 어떻게 되는지 기재되어 있다.
버튼 이벤트 내용	비활성화, 활성화, 선택했을 때 액션에 대한 내용이 기재되어 있다.
에러 상황	사용자가 올바르게 않은 액션을 했을 때 어떻게 처리할지 기재되어 있다.
제한 개수	데이터 제한 개수가 설정되어 있다.
외부 액션	데이터를 외부 수단(문자/이메일/카톡)으로 콘택트하는 시기/상황/메시지를 설계한다.
로직 내용	데이터가 나열되었을 때 표시되는 로직을 기재한다. (추천/최신/인기)

2. 화면 설계서

(1) 화면 설계서 유형

파워포인트(Powerpoint), 스토리보드(Storyboard), 와이어프레임(Wireframe), 목업(Mockup) 등이 있다.

(가) 와이어프레임: 기획 단계에서 페이지 레이아웃이나 화면 이동, 구성요소에 대한 내용을 기술한 문서이다.

(나) 스토리보드: 와이어프레임의 내용에 디스크립션 등을 포함한 설계 문서이다.

(다) 목업: 설계 단계에서 실제 화면과 같은 형태의 모형이다.

(2) 프로토타입(Prototype)

인터랙션(Interaction, 상호작용)이 포함되어 테스트가 가능한 형태이다.

(3) UI 디자인

화면의 모양이나 기능 등을 표현하는 것이다.

재료 · 자료

- 요구사항 정의서, 스크립트 설계서, 기능 명세 설계서
- 업무 정의서(업무 프로세스, 업무 처리 기준 등)

기기(장비 · 공구)

- 전산 장비(컴퓨터, 프린터, 빔 프로젝터, 네트워크 장비 등)
- OA 도구: 워드 프로세서, 스프레드시트, 프레젠테이션 도구 등

안전 · 유의 사항

- 설계를 지원하는 도구들을 다룰 수 있어야 한다.
- 프로그램 개발 경험과 오픈소스 경험이 필요하다.

수행 순서

① 기능 정의를 분석한다.

1. 기능 설계서를 분석한다.

(1) 요구사항 기능 설계서를 분석한다.

(가) 기능 요구사항을 확인한다.

시스템에 주어지는 특정 입력에 대한 시스템의 출력이 정의된다.

- 1) 시스템이 어떤 서비스를 제공하는지에 대한 요구사항을 확인한다.
- 2) 어떤 입력이 주어졌을 때 어떻게 반응하는지 확인한다.
- 3) 어떤 상황에서 어떻게 행동하는지 확인한다.
- 4) 시스템이 무엇을 해야 하는지에 대한 파악한다.

(나) 비기능 요구사항을 확인한다.

소프트웨어 기능들에 대한 조건과 제약사항에 관한 요구사항을 정의한다.

- 1) 기능적 요구사항보다 더 결정적인 부분이 될 수 있는 부분을 확인한다.
- 2) 각각의 특징과 서비스보다는 전체적인 시스템에 적용될 수 있는 것을 확인한다.

(2) function 설계 분석한다.

(가) 조건부로 생성될 수 있는 함수를 파악한다.

(나) 함수선언으로 끌어올릴 수 있는 부분을 파악한다.

2. 인터페이스 설계서를 분석한다.

(1) 최대처리 건수를 확인한다.

단위시간당 처리될 수 있는 해당 인터페이스 최대 수행 건수를 확인한다.

(2) 데이터 크기(평균/최대)를 확인한다.

해당 인터페이스 1회 처리 시 소요되는 데이터의 평균 크기와 최대 크기를 확인한다.

(3) 시스템 정보를 확인한다.

(가) 업무명을 확인한다.

인터페이스를 활용하는 업무명을 확인한다.

(나) 서비스명을 확인한다.

서비스 또는 프로그램을 호출하는 경우 해당 서비스명을 확인한다.

(다) 연계 방식을 확인한다.

웹서비스, FTP, DB Link, socket 등 아키텍처에서 정의한 방법을 확인한다.

(4) 데이터 정보를 확인한다.

(가) 데이터 타입 및 크기를 확인한다.

데이터 항목의 데이터 값 유형 및 데이터 항목의 데이터 길이를 확인한다.

(나) 매핑 규칙을 확인한다.

송신 시스템과 수신 시스템의 항목이 단순 1:1 아니므로 별도의 로직을 확인한다.

3. 컴포넌트 설계 분석을 한다.

컴포넌트는 인터페이스와 명시적인 의존성을 가진 조합의 단위로서 독립적으로 전개될 수도 있으며 제3의 컴포넌트와 맞출 수도 있는 소프트웨어 부품이다.

(1) 패키지 관점을 분석한다.

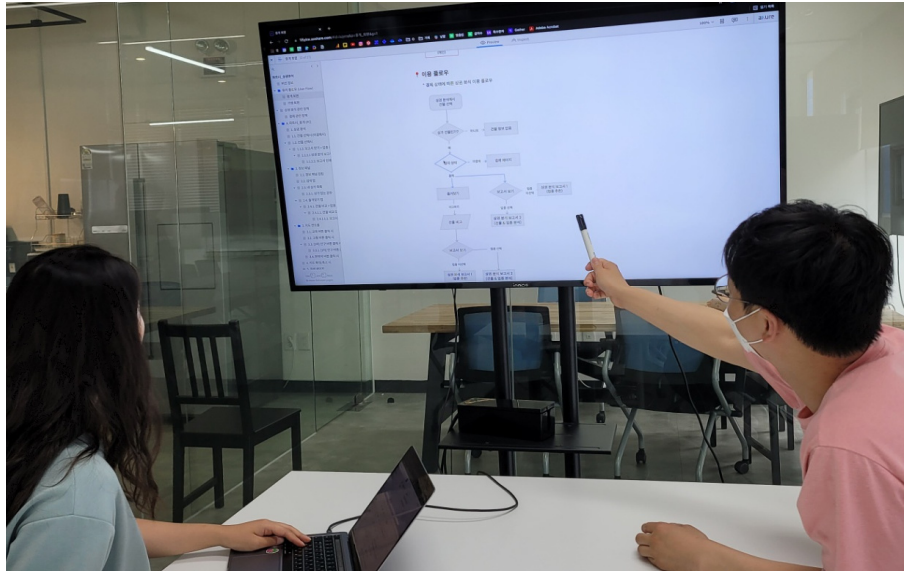
물리적으로 교체될 수 있고 재사용 가능한 단위로 분석한다.

(2) 서비스 관점을 분석한다.

서비스 제공자와 공급자 사이에서의 계약 관점으로 분석한다.

(3) 완전성 관점을 분석한다.

다른 컴포넌트에 대한 독립성 단위로 분석한다.



출처: 집필진 제작(2021)
[그림 3-1] 기능 설계서를 분석하는 모습

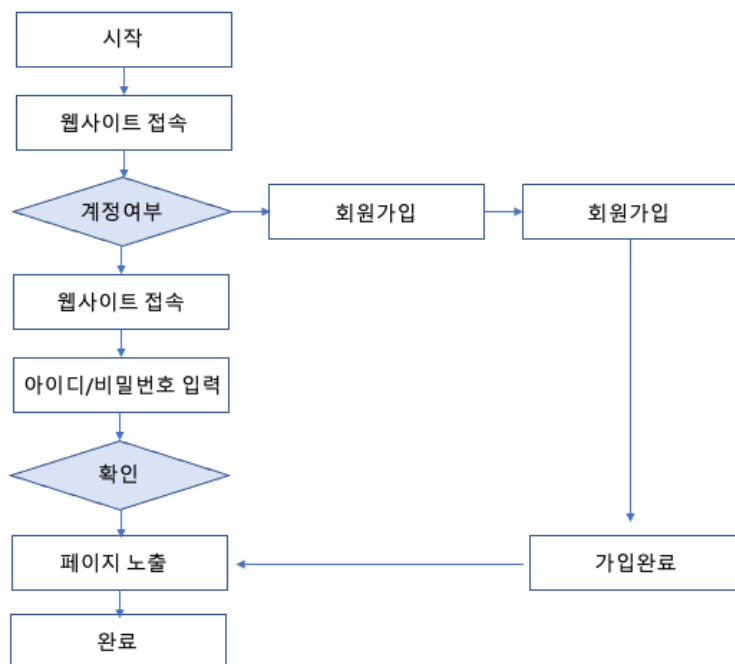
② 화면설계서를 분석한다.

다음의 스크립트 언어 설계서를 이해할 수 있어야 한다.

1. 플로차트를 분석한다.

업무 흐름에 따르는 플로차트 및 화면의 변화를 분석한다.

로그인



출처: 집필진 제작(2021)
[그림 3-2] 로그인 플로차트 예시

2. 기본 정책을 분석한다.

플로차트와 정책은 세트로 묶어서 한 흐름에 분석한다. 플로차트에서 언급되지 않은 내용은 정책 문서로 따로 정리되어 있으므로 참고한다.

아이템	* 파트너(B2B)의 경우, 직접 등록한다. [임시 저장 기능] * 아이템(브랜드/상가) 등록 시 각 스텝마다 임시 저장 가능 * 임시 저장된 아이템은 관리 페이지에서 보이며, 클릭 시 이어서 등록 가능 * 임시 저장된 아이템은 7일간 보관 후 삭제	
브랜드	* 가맹회원이 직접 등록하는 브랜드 (정보공개서 포함) * 공정거래위원회에 가맹 사업이 등록된 브랜드 * 브랜드 등록 번호 : 정보공개서 등록번호 (추후 브랜드 유형 2가지로 분류) * 파트너 브랜드 : 가맹회원이 직접 등록하는 브랜드 (정보공개서 포함) * 비파트너 브랜드 : 정보공개서만 올라온다.	
	정렬 기준	1. 수정 요청 2. 승인 완료 3. 검수 중 (가상 창업 오픈 시점에 건적서 사라짐으로 인한, 건적 발송 상태 삭제)
상가	* 중개회원이 직접 등록하는 상가 * 상가 등록 번호 : 어드민 상가 등록 번호	
	정렬 기준	1. 수정 요청 2. 승인 완료 3. 검수 중 4. 거래 완료 상태 (가상 창업 오픈 시점에 건적서 사라짐으로 인한, 건적 발송 상태 삭제)
	거래 상태	* 거래 대기 : 상가 거래 가능 상태 * 거래 완료 : 상가 거래 완료(불가능) 상태
	복제 기능	* 등록된 상가를 복제할 수 있는 기능 * 입력한 상가 정보가 복제되어 수정 후 등록할 수 있음 > 수정 페이지로 이동
정보공개서	* 공정거래위원회 가맹사업거래에서 가져오는 정보공개서	

출처: 집필진 제작(2021)
[그림 3-3] 정책 정의서 예시

3. 권한 정책을 분석한다.

사용자의 권한 및 보안상에 대하여 권한 정책을 보며 정리한다.

4. UI를 분석한다.

화면의 기능 및 화면의 전화에 대하여 UI를 보면서 분석한다.

출처: 집필진 제작(2021)
[그림 3-4] UI 설계서 예시

5. 유효성 검사를 판단한다.

보통 필수 입력 항목과 유효성 체크 시나리오가 정리되어 있으나 때로는 정리가 되어 있지 않을 경우가 있다. 이런 때에는 여러 가지 제약 사항을 판단하여 개발자가 정리한다.

수행 tip

- 소프트웨어 요구사항을 명확히 파악하고 컴포넌트에 대한 이해가 있어야 프로그램을 설계할 수 있다.

3-2. 스크립트 언어 활용

학습 목표

- 스크립트 언어를 활용하여 애플리케이션을 작성할 수 있다.
- 작성된 애플리케이션의 오류를 식별하고 수정할 수 있다.

필요 지식 /

① 스크립트 언어의 유형과 원리

1. 스크립트 프로그래밍 언어 유형

(1) JavaScript (자바스크립트)

미국 넷스케이프사가 개발한 스크립트 프로그래밍 언어이다. 자바스크립트는 표준 HTML 문서 내에 삽입되어 사용되며 인터랙티브한 웹 페이지를 만들 수 있다.

(2) jQuery

HTML 문서에 삽입하여 사용할 수 있는 자바스크립트 라이브러리이다.

(3) JSP (JavaServer Pages)

HTML 웹 페이지 클라이언트에 자바 코드를 직접 삽입하여 웹 서버에서 동적으로 웹 페이지를 생성하여 웹 브라우저에서 표현할 수 있도록 전달해주는 스크립트 프로그래밍 언어이다.

(4) PHP (Hypertext Preprocessor)

HTML 문서 안에 포함하여 작동하는 서버 측 스크립트 프로그래밍 언어이다. 리눅스 운영체제에 아파치 웹 서버를 설치하고 MySQL DB 환경에서 주로 같이 사용된다.

(5) ASP (Active Server Pages)

마이크로소프트사에서 개발된 윈도 서버에서 운영할 수 있도록 개발된 스크립트 방식의 웹 프로그래밍 언어이다.

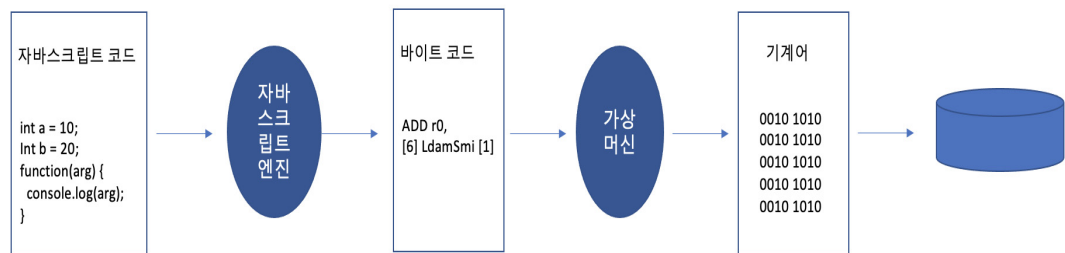
(6) Python

객체지향 스크립트 프로그래밍 언어로 별도의 컴파일 과정이 필요 없이 개발되어 개발 기간이 단축되고, 리눅스나 윈도 등 특정 플랫폼에 의존적이지 않고 독립적이다.

(7) VBScript

미국 마이크로소프트 Visual Basic(비주얼베이직) 기반의 스크립트 프로그래밍 언어이다.

2. 스크립트 프로그래밍 원리



출처: 집필진 제작(2021)

[그림 3-5] 스크립트 프로그래밍 원리

(1) 스크립트 코드 작성

최초 스크립트 코드를 작성한다.

(2) Byte Code 변환

스크립트 엔진은 위에서부터 한 줄씩 해석하면서 가상 머신이 이해할 수 있는 Byte Code로 변환한다.

(3) 기계어 변환

가상 머신은 Byte Code를 CPU가 이해할 수 있는 기계어로 변환한다. 단, CPU마다 기계어를 다르게 해석하므로 가상 머신은 CPU별로 최적화된 기계어를 만들어낸다.

(4) CPU 코드 변환

컴퓨터의 CPU는 가상 머신이 만들어낸 기계어를 수행한다. 기계어대로 메모리에 데이터를 저장하고, 읽고, 연산하고, 출력하는 작업 등을 수행한다.

② JavaScript 언어의 이해

자바스크립트는 객체지향의 프로그래밍 언어로서 웹 브라우저에서 주로 사용된다. 자바스크립트를 이용하여 웹 페이지에서 발생하는 사용자 이벤트에 대한 처리가 가능하고, 자바스크립트의 내장 객체를 활용하면 다양한 형태의 웹 페이지를 구현할 수 있다.

1. 자바스크립트 구현 예

아래 자바스크립트 구현 예제를 보면 자바스크립트를 이용하여 HTML 페이지의 요소를 변경하는 것을 확인할 수 있다.

자바스크립트 구현 예시

```
<html>
<head>
  <script>
    function myFuction() {
      document.getElementById("exam").innerHTML = "변경 후"
    }
  </script>
</head>
<body>
  <h1> JavaScript init </h1>
  <p id="exam"> 변경 전 </p>
  <button type="button" onclick="myFunction()">변경</button>
</body>
</html>
```

2. 자바스크립트 작성 방법

자바스크립트는 HTML 문서 내에서 태그를 통해 작성되고, 작성되는 위치는 스크립트 태그 영역이며, .js 확장자를 갖는 외부 파일 형태로 작성할 수 있다.

- (1) 영역 내부의 태그 내부에 자바스크립트 코딩을 작성한다.
- (2) .js 확장자를 갖는 별도의 외부 자바스크립트 코드 파일을 생성하고, 이를 HTML 페이지에 적용할 수 있다. 이 방식의 장점은 동일한 자바스크립트 코드를 여러 HTML 페이지에 적용할 수 있다는 것이다.

HTML 기본 구성

```
<html>
<body>
  <script src="mainScript.js"></script>
</body>
</html>
```

수행 내용 / 자바스크립트를 이용한 애플리케이션 개발하기

재료 · 자료

- 요구사항 정의서
- 화면 설계서
- 기능 사항 정의서
- 업무 정의서(업무 프로세스, 업무 처리 기준 등)
- IEEE 830(IEEE Recommended Practice for Software Requirements Specifications)

기기(장비 · 공구)

- 전산 장비(컴퓨터, 프린터, 빔 프로젝터, 네트워크 장비 등)
- OA 도구: 워드 프로세서, 스프레드시트, 프레젠테이션 도구 등
- CASE 도구: 요구사항 관리 도구, SW 분석 지원 도구, 모델링 도구, 형상관리 도구 등
- web OS 디버깅 도구

안전 · 유의 사항

- 다이어그램 저작 도구는 라이선스 제약이 없는 오픈소스 저작 도구를 활용한다.
- 설계를 지원하는 도구들을 다룰 수 있어야 한다.

수행 순서

① 화면을 구현한다.

1. JavaScript를 작성한다.

JavaScript 언어는 객체 기반의 스크립트 언어로, HTML로는 웹의 보여주는 내용을 작성하고, CSS로는 웹의 동적인 요소를 디자인하며, 자바스크립트로는 웹의 동작을 구현하는 역할을 한다.

자바스크립트 예시

```
<!DOCTYPE html>
<html lang="ko">

<head>
  <meta charset="UTF-8">
  <title>JavaScript Intro</title>
  <script>
    document.write("<h2>여러분을 환영합니다!!</h2>")
  </script>
</head>

<body>
  <noscript>
    <p>여러분의 웹 브라우저가 자바스크립트를 지원하지 않습니다!</p>
  </noscript>
</body>

</html>
```

(1) 출력을 구현한다.

- (가) window.alert() 메소드를 이용한 출력을 할 수 있다.
- (나) HTML DOM 요소로 innerHTML 프로퍼티를 이용한 출력을 할 수 있다.
- (다) document.write 메소드를 이용한 출력을 할 수 있다.
- (라) console.log 메소드를 이용한 출력을 할 수 있다.

(2) 선언을 구현한다.

(가) 숫자(Number)

자바스크립트는 다른 언어와는 다르게 실수와 정수를 따로 구분하지 않고, 모든 수를 실수 하나로만 표현한다. 또한, 매우 작은 수나 매우 큰 수를 표현할 경우에는 e 표기법을 사용할 수 있다.

(나) 문자열(String)

자바스크립트에서 문자열은 작은따옴표(')나 큰따옴표("")로 둘러싸인 문자의 집합이다. 작은따옴표는 큰따옴표로 둘러싸인 문자열에만 포함될 수 있으며, 큰따옴표는 작은따옴표로 둘러싸인 문자열에만 포함될 수 있다.

(다) Boolean

Boolean은 참(True)과 거짓(False)을 표현하는 문구로 자바스크립트에서 Boolean값은 예약어인 true와 false를 사용한다.

② 스크립트 언어로 개발한다.

1. 변수 설정을 한다.

(1) 전역변수를 선언한다.

어떤 함수에도 속하지 않은 상태에서 선언되거나, 함수에 속하더라도 특정 키워드를 통해 선언되지 않은 채로 사용되는 변수이다.

(2) 지역변수를 선언한다.

지역변수는 해당 지역에서 선언됨과 동시에 메모리가 생성되고 해당 지역을 나오면 자동으로 소멸된다.

2. 함수를 적용한다.

(1) 함수를 적용한다.

함수는 하나의 특정 기능을 수행하도록 설계된 독립적인 블록이다.

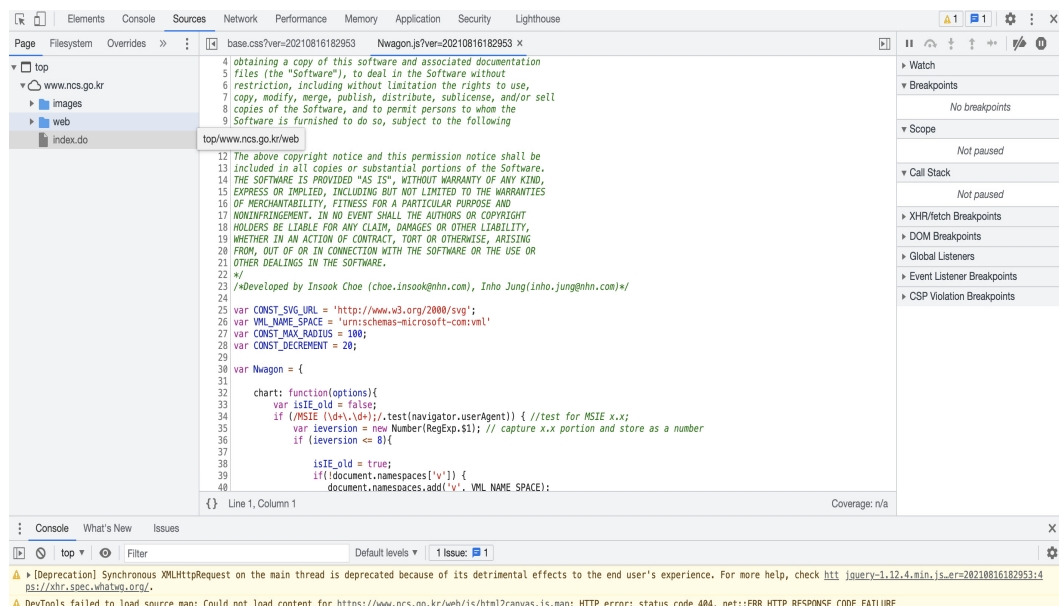
(2) 반환문을 고려한다.

함수의 실행을 중단하고, return 다음에 명시된 표현식의 값을 호출자에게 반환한다. 반환문은 배열이나 객체를 포함한 모든 타입의 값을 반환할 수 있다.

③ 디버깅으로 오류를 확인한다.

해당 디버깅 사례는 크롬 브라우저 기준으로 설명한다.

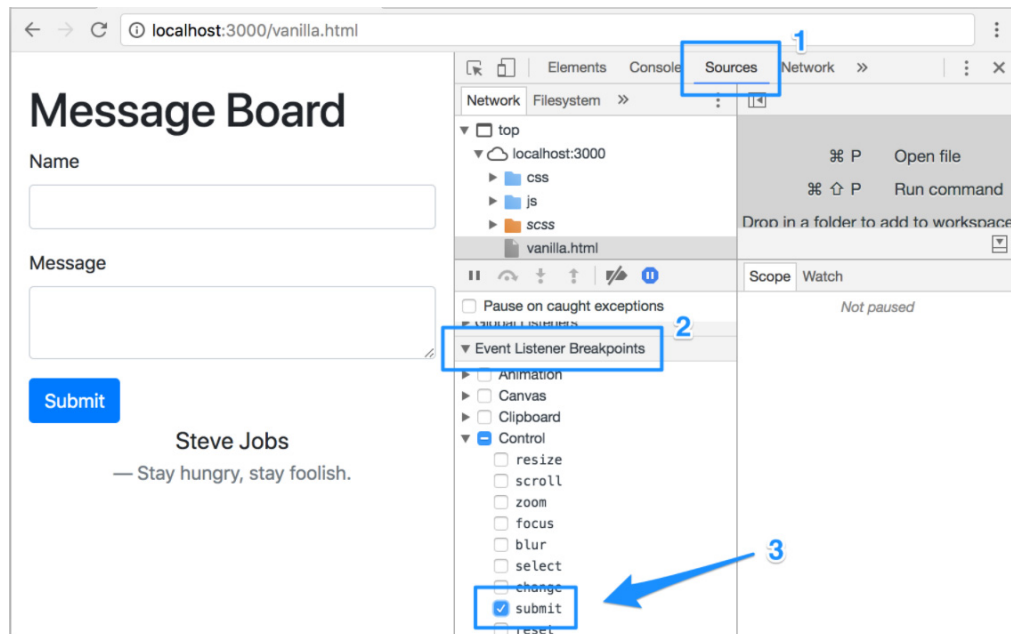
1. 개발자 도구를 연다.



출처: 집필진 제작(2021)

[그림 3-6] JavaScript 디버깅 화면

2. 중단점을 설정한다.



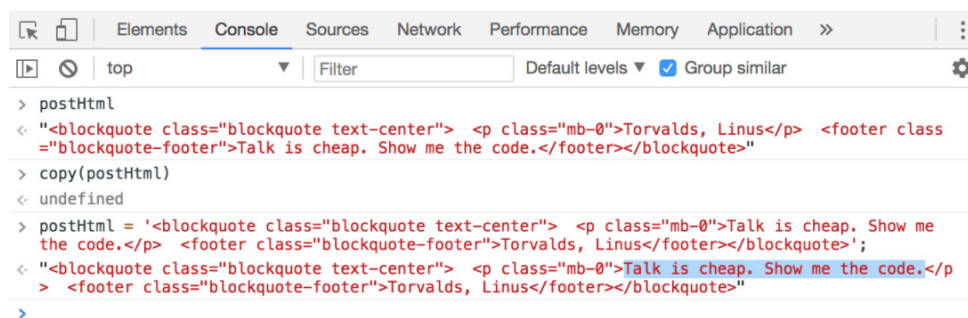
출처: 집필진 제작(2021)
[그림 3-7] JavaScript 디버깅 화면

3. 단계별로 코드를 실행한다.

코드를 한 줄씩 단계별로 실행하면서 스크립트가 어떻게 실행되는지, 변수가 어떻게 저장되어 있는지 확인해 본다.

4. 콘솔에서 테스트한다.

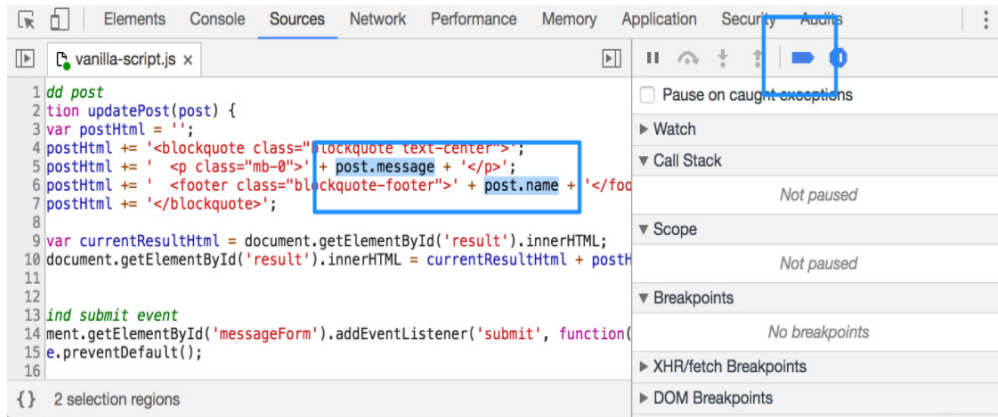
콘솔을 확인하여 테스트를 한다.



출처: 집필진 제작(2021)
[그림 3-8] JavaScript 테스트 화면

5. 소스를 수정한다.

오류가 발생한 부분에 대하여 소스를 수정한다.



출처: 집필진 제작(2021)

[그림 3-9] JavaScript 오류 수정

수행 tip

- 스크립트 표준을 숙지하여 개발 활용에 이용하도록 한다.

교수 방법

- 스크립트 설계 내용을 개발 언어와 도구를 활용하여 구체적으로 설명한다.
- 스크립트 개발 시 개발 생산성을 높이기 위한 도구를 제시하여 효율적인 개발을 할 수 있도록 도구 활용 방안에 대해 설명한다.
- 스크립트 개발 시 오류를 수정할 수 있는 디버깅 기술에 대하여 정리하여 설명한다.
- 개발 모듈의 단위 테스트 수행을 위한 테스트 조건 명세화 절차와 내용에 대해 정리한 후 상세히 설명한다.

학습 방법

- 스크립트 프로그램 설계 내용을 개발 언어와 도구를 활용하여 개발하는 절차에 대해 여러 참고 자료를 활용하여 학습한다.
- 스크립트 프로그램 개발 시 개발 생산성을 높이기 위한 도구를 제시하여 효율적인 개발을 할 수 있도록 실습한다.
- 개발된 스크립트 프로그램에서 오류에 대해 조치할 방법을 학습하여, 이를 실무에 활용할 수 있을 정도로 반복적으로 실습한다.
- 개발 모듈의 단위 테스트 수행을 위한 테스트 조건 명세화 절차와 내용에 대해 학습한다.

평가 준거

- 평가자는 학습자가 학습 목표를 성공적으로 달성하였는지를 평가해야 한다.
- 평가자는 다음 사항을 평가해야 한다.

학습 내용	학습 목표	성취수준		
		상	중	하
스크립트 언어 설계	- 응용소프트웨어 개발을 위하여 프로그램 설계서를 확인할 수 있다.			
스크립트 언어 활용	- 스크립트 언어를 활용하여 애플리케이션을 작성할 수 있다.			
	- 작성된 애플리케이션의 오류를 식별하고 수정할 수 있다.			

평가 방법

- 평가자 체크리스트

학습 내용	평가 항목	성취수준		
		상	중	하
스크립트 언어 설계	- 응용소프트웨어 개발 프로그램 설계서 파악 능력			
	- 스크립트 개발 요구사항 도출 능력			
스크립트 언어 활용	- 스크립트 언어를 활용한 애플리케이션 기능 개발 능력			
	- 작성된 애플리케이션의 오류 식별 및 수정 능력			

• 포트폴리오

학습 내용	평가 항목	성취수준		
		상	중	하
스크립트 언어 설계	- 응용소프트웨어 개발 프로그램 설계서 기능 도출 능력			
	- 스크립트 기능 도출 능력			
스크립트 언어 활용	- 스크립트 언어를 활용한 애플리케이션 작성 능력			
	- 작성된 애플리케이션 오류 식별 및 수정 능력			

피드백

1. 평가자 체크리스트

- 실습 과정을 체크리스트에 따라 평가한 후에 개선해야 할 사항 등을 정리한 후 학습자에게 설명한다.
- 설계서를 분석하여 작성된 개발 모듈이 설계서대로 개발되었는지 확인하여 비정상 동작 시 그 문제점을 정리하여 개선 사항을 설명해준다.
- 스크립트 언어 특징에 대한 부족한 이해도를 보충할 수 있도록 실무 자료를 제시한다.
- 다양한 설계도를 보여주면서 설계 이해 경험을 제시해준다.

2. 포트폴리오

- 제출한 보고서를 평가한 후에 주요 사항에 대하여 의견을 제시한다.
- 개발 언어와 도구의 활용을 통하여 정상적으로 동작하는지를 보고 비정상 동작의 원인과 조치 방안에 대해 설명한다.
- 개발 언어 문법의 오류를 확인하여 개선사항을 설명한다.
- 스크립트 디버깅 방법에 대해 정확히 이해하고 있는지 확인하여 미흡할 경우 디버깅 방법을 명확하게 설명해준다.

참고 자료



- 교육부(2017). 기능모델링(2001020220_19v2). 한국직업능력개발원.
- 이형우(2019). 『모두의 C언어』. 길벗.

NCS학습모듈 개발이력

발행일	2021년 12월 31일		
세분류명	응용SW엔지니어링(20010202)		
개발기관	(사)한국정보통신기술사협회(개발책임자: 온기현), 한국직업능력연구원		
집필진	김승환(㈜캐롯아이)*	안응원(미라콤아이앤씨)	
	김원기(LG CNS)	유현주(한국전산감리원)	
	박주형(㈜오아시스비즈니스)	검토진	홍필두(한국폴리텍대학)
	엄기영(우리FIS)		
	윤혜경(만도)		
	최홍선(㈜에이스기술단)	*표시는 대표집필자임	

프로그래밍 언어 활용(LM2001020231_19v4)

저작권자	교육부
연구기관	한국직업능력연구원
발행일	2021. 12. 31.
ISBN	979-11-339-9444-1

※ 이 학습모듈은 자격기본법 시행령(제8조 국가직무능력표준의 활용)에 의거하여 개발하였으며, NCS통합포털사이트(<http://www.ncs.go.kr>)에서 다운로드 할 수 있습니다.



www.ncs.go.kr